Desarrollo de Aplicaciones Web y Multiplataforma: **Entornos de Desarrollo**

DOCENTE: Daniel López Lozano

atlántida
CIDEP
Centro de Investigación y Desarrollo de Estudios Profesionales

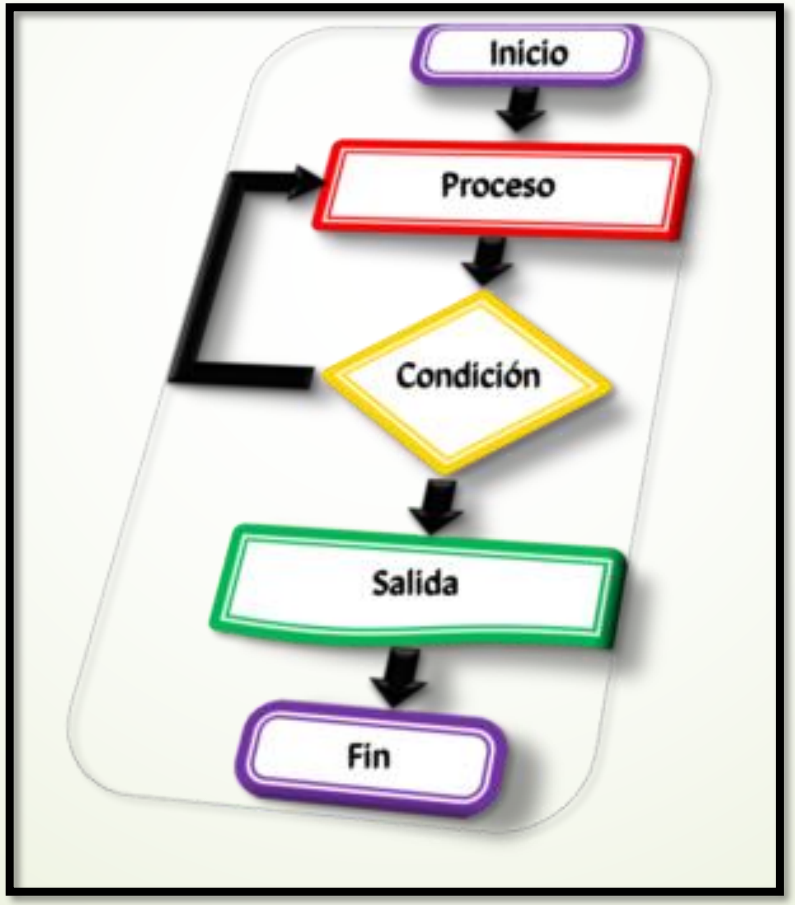
Tema 1. Introducción al Desarrollo Software

Índice de contenidos

- Conceptos básicos.
- Los lenguajes de programación. El lenguaje Java.
- Conceptos de Ingeniería del software.
  - Proceso de desarrollo.
  - Ciclo de vida del Software.
  - Metodologías Agiles.
  - Scrum, DevOps, CI/CD

Algoritmo (RAE)

- Conjunto ordenado y finito de operaciones que permite hallar la solución de un problema.



Algoritmo
(Introducción a la Informática Mc-Graw Hill)

- Es un **conjunto ordenado** de **reglas o instrucciones** tal que siguiéndolas paso a paso se obtiene la **respuesta** a un **problema** dado.

- **Sean cuales sean** los datos o circunstancias particulares del mismo.

Algoritmo (Wikipedia)

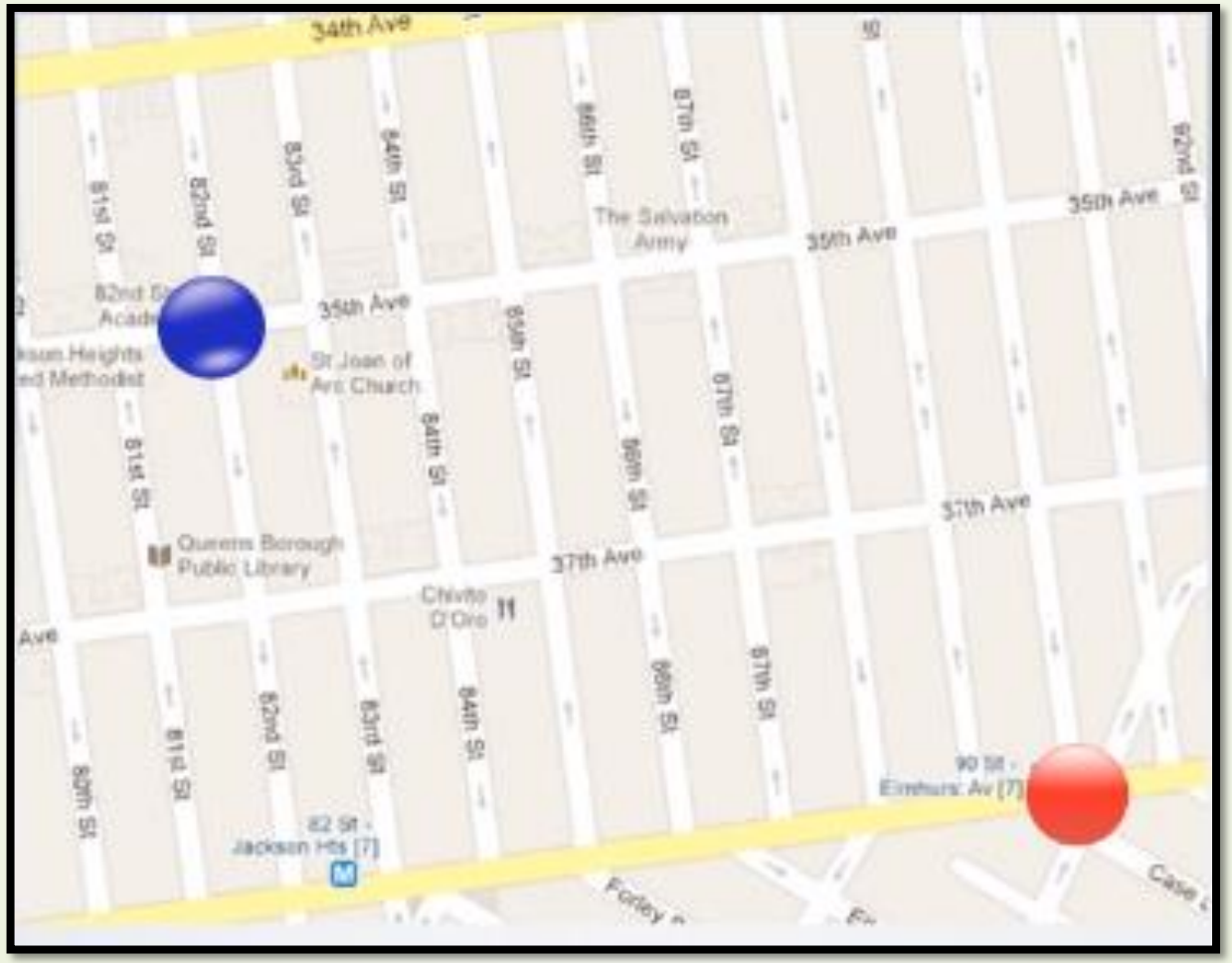
- Es un conjunto de instrucciones o **reglas bien definidas**, **ordenadas y finitas** que permite **realizar una actividad** mediante pasos sucesivos que **no generen** dudas a quien deba realizar dicha actividad.

- Dados un **estado inicial y una entrada**, siguiendo unos pasos sucesivos se llega a un **estado final** y se obtiene una solución.

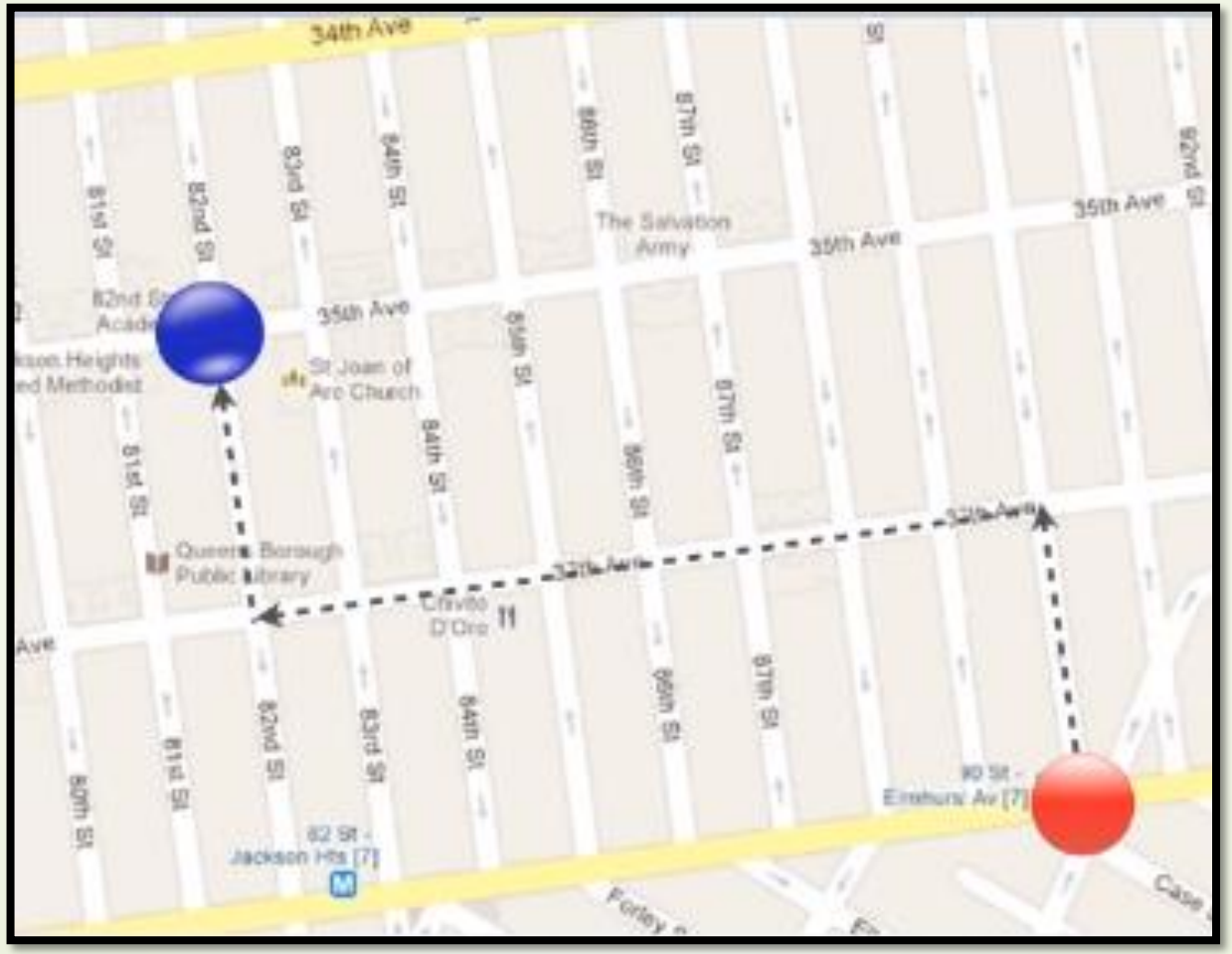
Algoritmo

CONCRETO
LEGIBLE
FINITO
Debe generar el mismo resultado siempre que se siga.
EFICIENTE
DEFINIDO
NO AMBIGUO
PRECISO

La vida como un algoritmo



La vida como un algoritmo



Analogía

- Pensarás que es algo con lo que no estamos familiarizados, pero es todo lo contrario.

- **¿Cómo se hace una suma de varias cifras?**

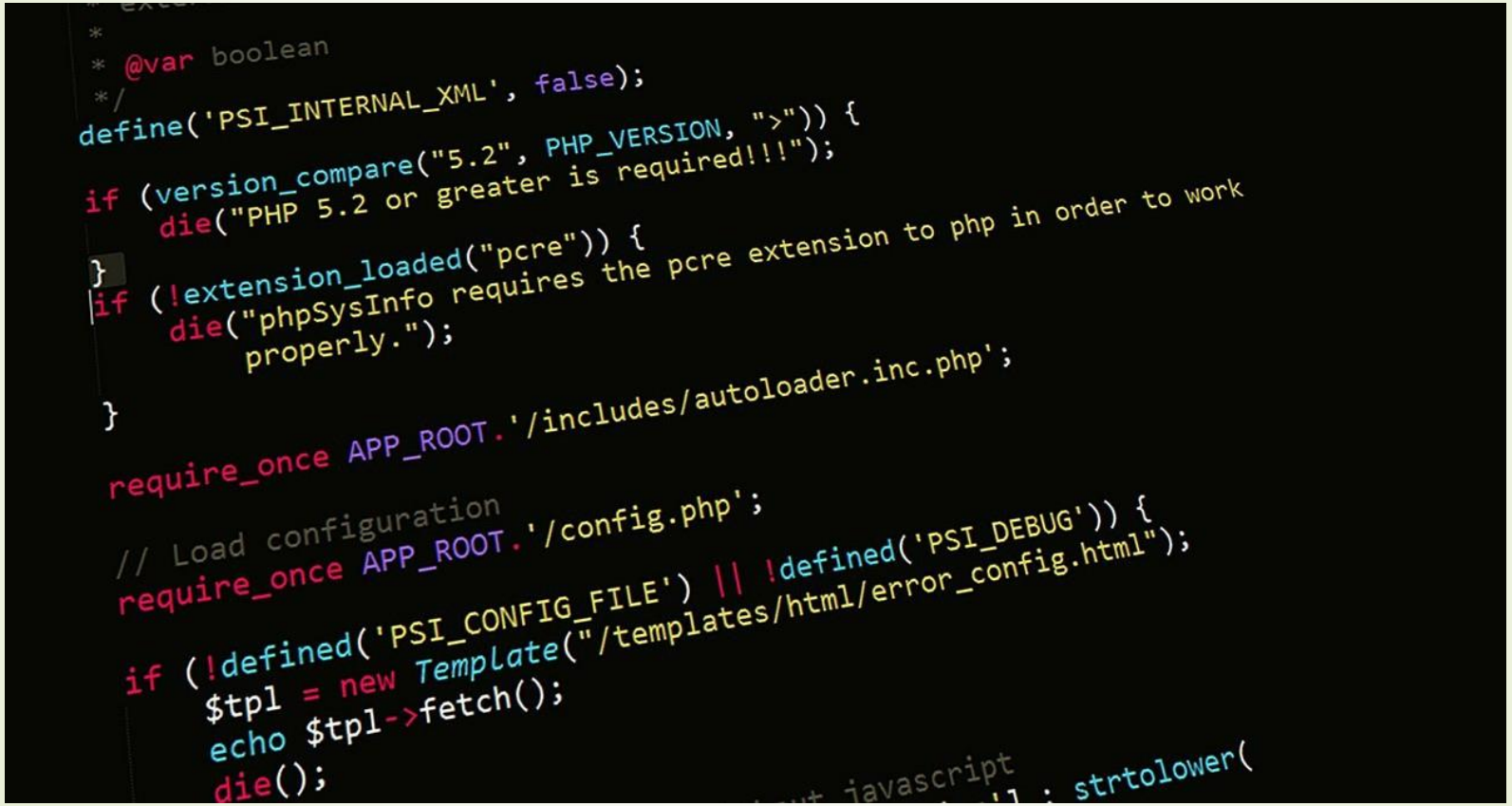
- **¿Cómo cocinar un huevo frito?**

- **¿Cómo se hace un cola-cao?**

- **¿Cómo se hace una llamada?**

- **¿Cómo se hace una tortilla de patatas?**

Programa informático



Programa informático

- Es un conjunto ordenado de instrucciones **que se dan a la computadora** indicándole las operaciones o tareas que se desea realice. **(Mc-Graw Hill)**

- **Conjunto unitario** de instrucciones que permite a un ordenador realizar **funciones diversas**, como el tratamiento de textos, el diseño de gráficos, la resolución de problemas matemáticos, el manejo de bancos de datos, etc. **(RAE)**

¿Cuál es la diferencia entre algoritmo y programa informático?

- Los algoritmos **pueden ser expresados de muchas maneras** incluyendo el lenguaje natural, pseudocódigo, diagramas de flujo y lenguajes de programación entre otros.

- Mientras que los programas informáticos son la **implementación de un algoritmo** en un lenguaje de programación determinado para implantarse sobre una plataforma o plataformas determinadas.

Software

(Según el estándar 729 de IEEE)

- Conjunto de los programas de **cómputo, procedimientos, reglas, documentación y datos asociados** que forman parte de las operaciones de un sistema de computación.

- Este concepto **va más allá** de los programas de computación en sus distintos estados (**código fuente, binario o ejecutable**).

- El concepto abarca además **su documentación, los datos a procesar** e incluso la información que suministra **el usuario**.

- Clasificación de los programas o del software:
  - ✓ **Software de sistema.**
  - ✓ **Software de aplicación.**
  - ✓ **Software de programación.**

Lenguajes de programación

- Es un **idioma artificial** diseñado para expresar tareas que pueden **ser llevadas a cabo por máquinas** como las computadoras.

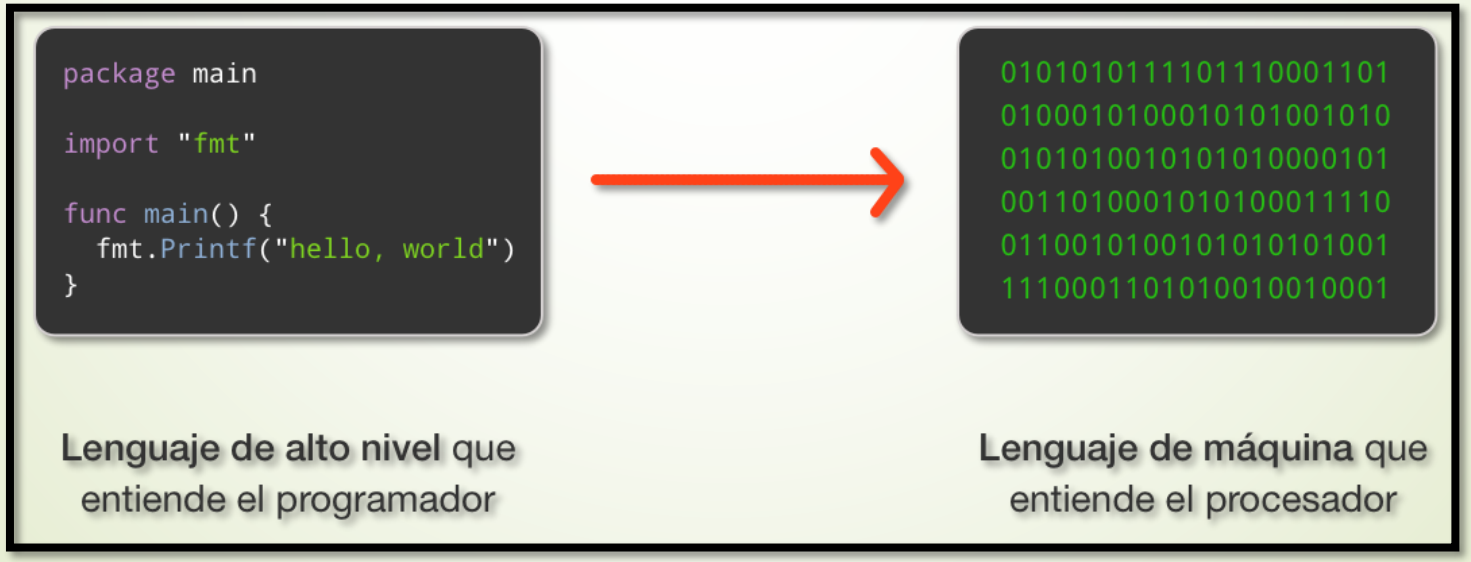
- Está formado por un conjunto de **reglas sintácticas y semánticas** que definen su estructura y el significado de sus elementos y expresiones.

- Permite especificar de **manera precisa** sobre que datos debe operar una computadora, como deben ser **almacenados o transmitidos**, y que **acciones debe tomar** bajo una determinada gama de circunstancias.

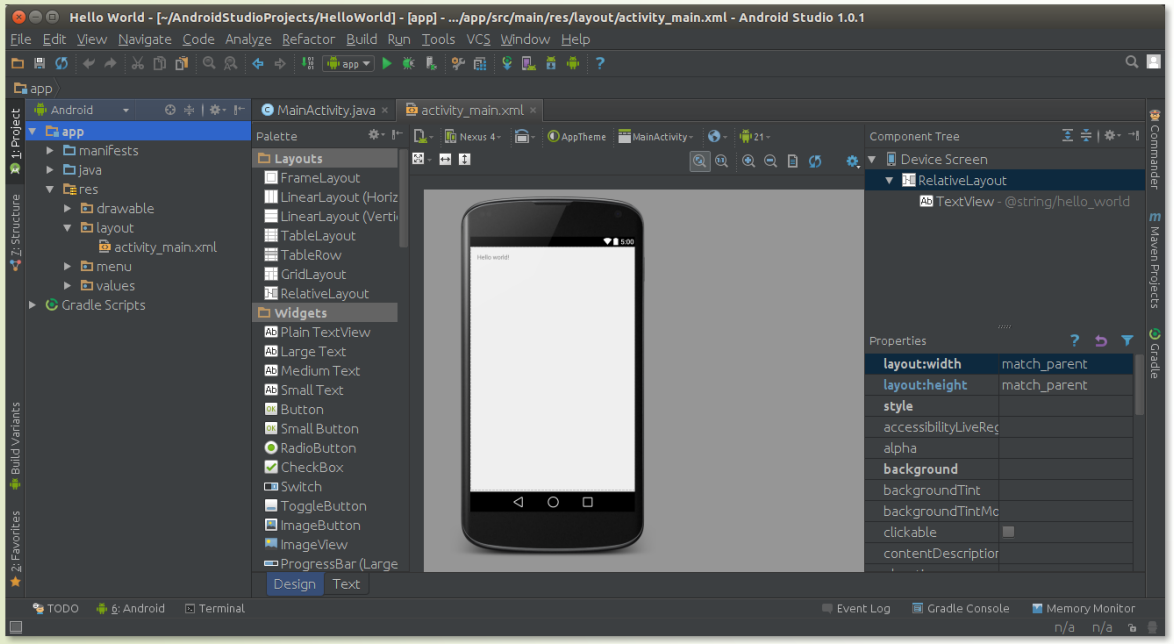
- Utilizaremos **3 criterios** para clasificar los lenguajes de programación:
  - ✓ **Nivel de abstracción.**
  - ✓ **Forma de ejecución.**
  - ✓ **Paradigma.**

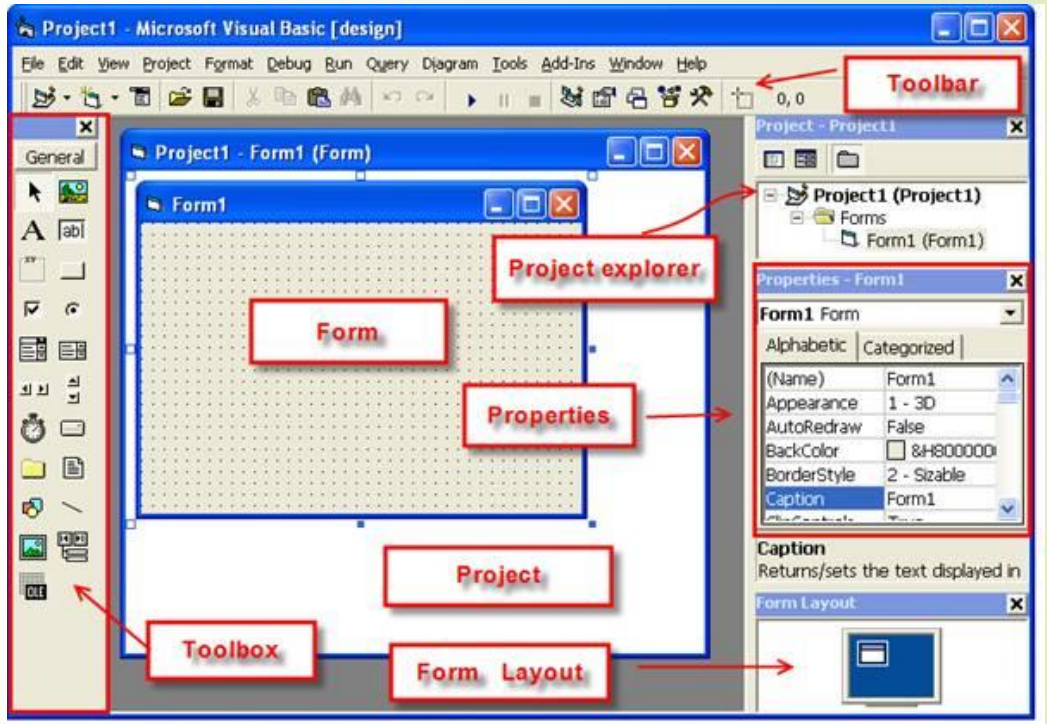
- El **nivel de abstracción** se refiere al modo en que los lenguajes de programación **se alejan del código maquina** cada más hasta un lenguaje **similar a un lenguaje humano**.

- Lenguaje máquina y de bajo nivel. **(Binario, ensamblador)**
- Lenguajes de medio nivel. **(PASCAL, ADA, C)**
- Lenguajes de alto nivel. **(C++, C#, JAVA)**
- Lenguajes de última generación. **(Visuales y de IA)**



Lenguajes Visuales





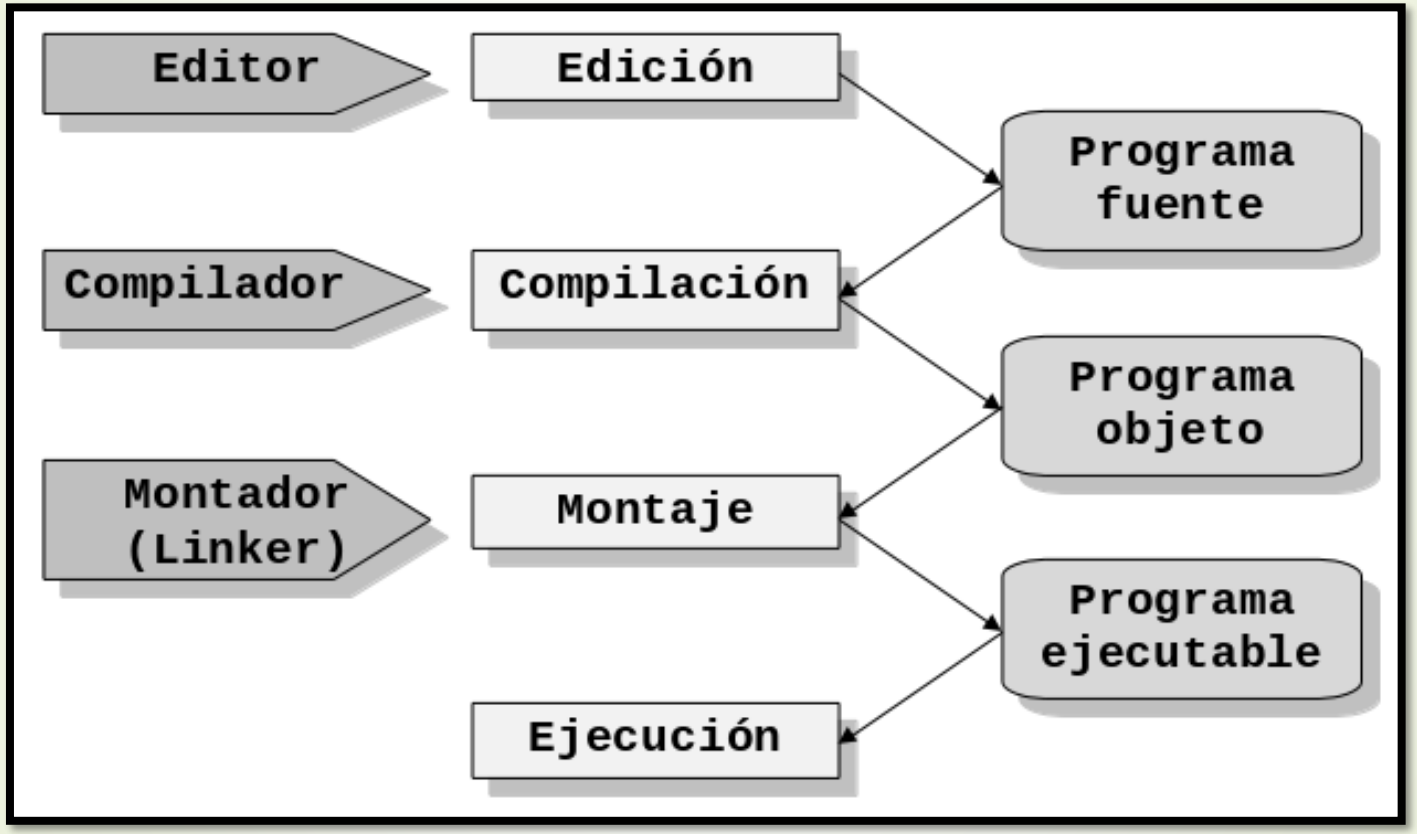
- Podemos encontrarnos 3 formas de **ejecutar programas** escritos en **lenguajes de alto nivel**.

- **Una vez escrito el código fuente** de un programa , este **se traduce** por medio de un **compilador** en un archivo **ejecutable** para una determinada plataforma.

- Los **lenguajes compilados** son lenguajes de programación en los que las instrucciones se traducen del lenguaje utilizado a código máquina para una ejecución rápida.

- **C, C++, Ada, Fortran , Pascal, Modula 2, etc.**

Proceso en los lenguajes compilados



- **Código fuente:** es un conjunto de instrucciones escritas en un determinado lenguaje de programación, es decir, es el código en el que **están escritos nuestros programas**.

- **Código objeto:** es el código **resultante de compilar** nuestro código fuente. En los lenguajes compilados seria lo que se denomina **código maquina**.

- **Código ejecutable:** es el resultado de **enlazar nuestro código** con las librerías que utilicemos, generándose un programa ejecutable para nuestro **sistema operativo**.

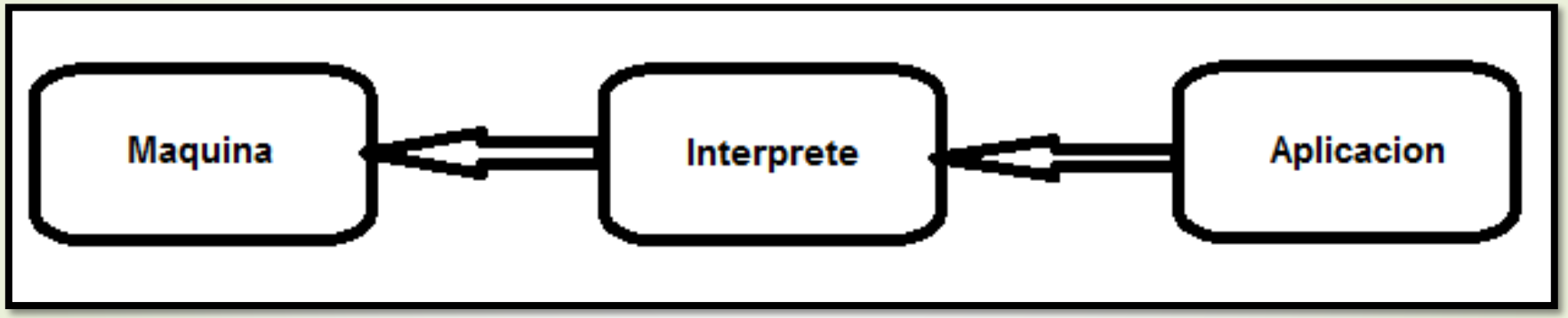
- Un **lenguaje de programación interpretado** es aquel en el que las instrucciones se traducen o interpretan **una a una**.

- Los lenguajes interpretados **son compilados línea por línea**; es decir cada línea del programa se compila cada vez que va a ser ejecutada, y si por algún motivo alguna línea se ejecuta más de una vez, se compila **tantas veces como fuera necesario**.

- **PHP, JavaScript, Perl, Lisp, Bash, etc**

Proceso en los lenguajes interpretados

- Puede parecer **un esquema simple**, pero suele generar programas **poco eficientes.**



- Tiene la **ventaja** de que los programas son **multiplataforma.**

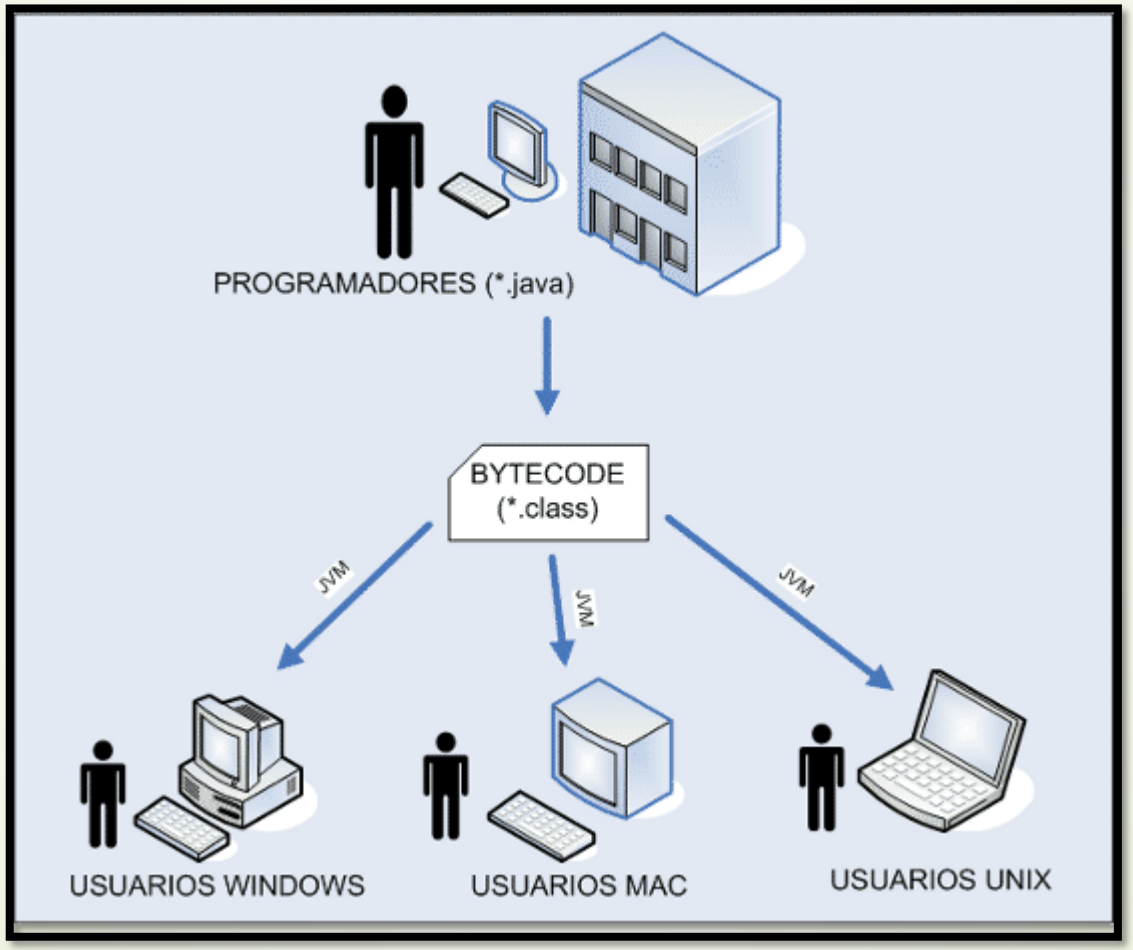
- Los lenguajes **semi-interpretados** están a camino entre los dos anteriores.

- Estos lenguajes utilizan una representación intermedia, que **combina** tanto la interpretación como la compilación.

- En este caso el compilador produce **una representación intermedia** del **programa o bytecode**, que después es ejecutado por **un interprete**.

- **Java, Python, Ruby, Scala, etc.**

Proceso en lenguajes semi-interpretados



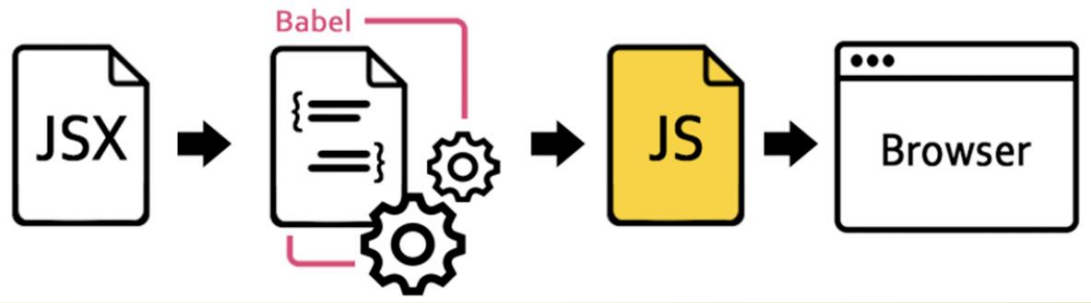
- Dependiendo de como se construya un programa existen un **conjunto de paradigmas**:

- Lenguajes procedimentales o imperativos.
  (C, C++, Pascal, Cobol, Batch (MS-DOS))
  - ✓ **Estructurados. (Usan bucles ni procedimientos)**
  - ✓ **No estructurados. (No usan bucles ni procedimientos)**
- Lenguajes declarativos o predicativos. **(SQL)**

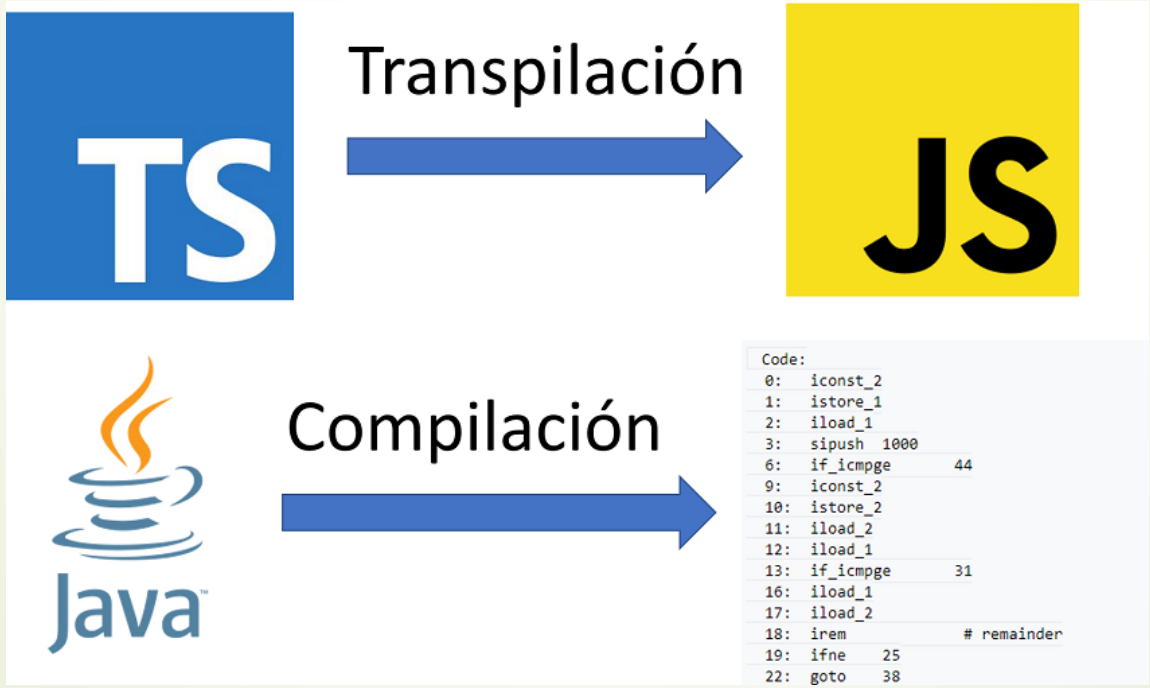
- Lenguajes orientados a objetos.
  (Java, JavaScript, Python, etc)

- Hoy dia con la evaluación de los lenguajes y estándares web existen nuevos tipos de traductores de lenguajes llamados transcompiladores.
- Este tipo de lenguajes que surgen de los transcompiladores transforman un lenguaje de alto nivel a otro también de alto y se usan para mantener compatibilidad y seguir funcionando con los lenguajes actuales.
- En este, grupo incluimos a SASS, LESS, HandleBars y el más importante hoy dia que es TypeScript.
- Estos lenguajes han elevado su importancia debido a la aceptación de los navegadores de los estándares web.

- Toda aplicación depende de HTML , CSS y JavaScript que después de muchos años son estándares en cualquier navegador y sistema operativo.
- Los lenguajes transcompilados o también llamados pre-procesados permite añadir nuevas funcionalidades sin necesidad de crear un lenguaje totalmente nuevo.



- Aunque realmente en ningún momento se genera código ejecutable debemos tener el compilador o interprete del lenguaje de base.
- En otras palabras no es código nativo



El lenguaje Java

- **Desarrollado** por los laboratorios **Sun**, (hoy en día pertenece a Oracle), es uno de los lenguajes de programación orientados a objetos de **mayor repercusión en los últimos años**.

- Se emplea tanto en la programación **tradicional, distribuida, GUI, Web, dispositivos móviles,** etc.

- Esta basado en **C++ pero simplificado**, por tanto es mucho más fácil de usar, de más alto nivel y menos propenso a errores.

Historia de Java

- En 1991 un grupo de ingenieros de Sun Microsystems liderados por **James Gosling** y **Patrick Naughton** tenían que desarrollar un lenguaje de programación nuevo.
- Dicho lenguaje de programación se debería poder utilizar para **pequeños electrodomésticos**.
- Por lo tanto debía ser **pequeño, compacto y neutro** respecto a la arquitectura.

- Que fuera neutro respecto de la arquitectura era fundamental debido a que si el **hardware cambia**, la tecnología de programación **no se viese afectada**.

- En cierto modo esta idea dio lugar a lo que hoy llamamos como el **internet de las cosas**.

- Tuvo un **fracaso comercial estrepitoso**. Ninguna empresa vio con buenos ojos la idea revolucionaria que traía el lenguaje.

- El proyecto no se cancelo completamente sino que se quedo en **stand-by**, por ello a la versión que usamos de Java se conoce como **Java2**.

- Con la llegada masiva de internet a partir del **1994 la situación cambio** porque las características de Java se adaptaban como un guante.

- Por ejemplo se creo un **navegador web** que permitía correr programas en Java.

- Estos programas son los denominados **Applets**, que hoy día no son tan populares pero fueron **revolucionarios** en su momento.

- La tecnología de **firma digital** de algunas aplicaciones web de la administración publica **funcionan con ellos**.

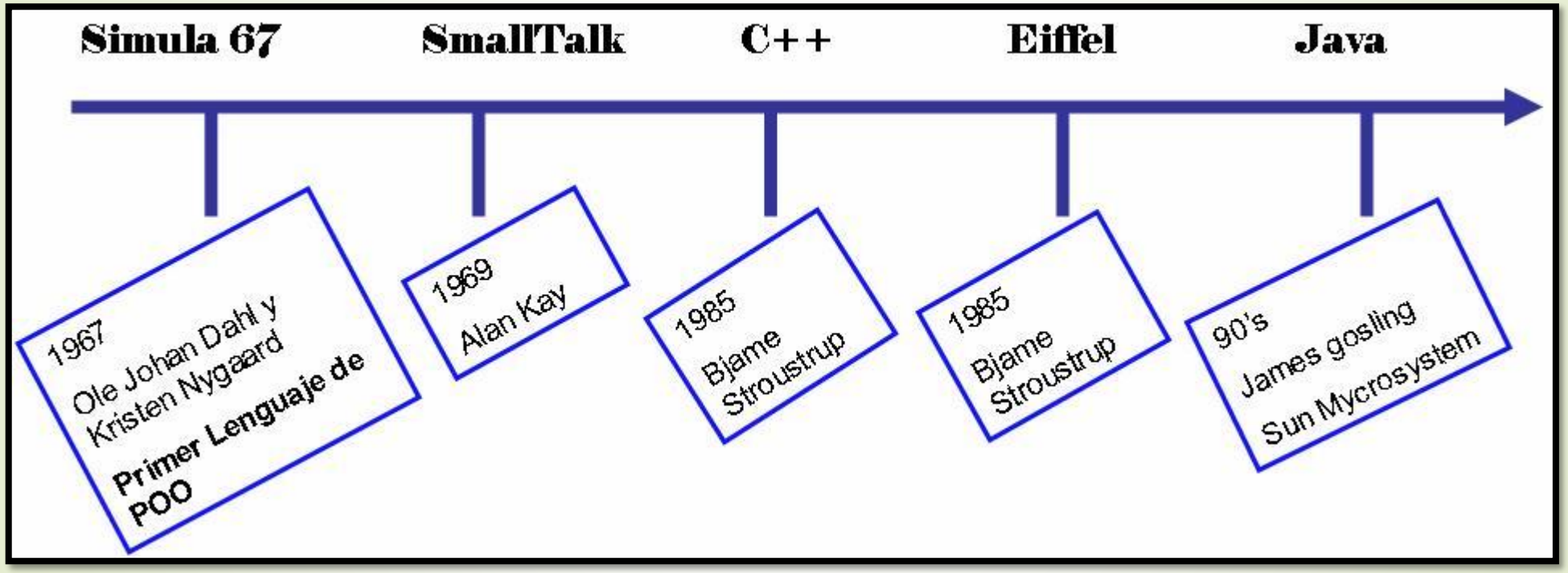
- **Sencillo en comparación** con otros lenguajes orientados a objetos eliminando características engorrosos como el manejo de memoria a través de punteros.

- **Orientado a objeto y distribuido** (aplicaciones para internet).

- Es **seguro**.

- Neutro y por lo tanto **multi-arquitectura** (Windows, Linux, Mac, Intel, AMD, etc).

- No fue ni mucho menos el primer lenguaje de programación orientado a objetos:



Ideas erróneas

- Java esta relacionado con **JavaScript**.
- Java es una extensión de **HTML**.
- Todo programa Java tiene que ejecutarse en un **página web**.
- Java es **inseguro**.
- Java es **100% seguro**.

- Posee una **amplísima biblioteca estándar** de clases predefinida.
- Las aplicaciones **Java** pueden ser ejecutadas indistintamente **en cualquier plataforma** sin necesidad de recompilación.
- Las aplicaciones están típicamente compiladas en un **bytecode**.
- En **tiempo de ejecución el bytecode** es interpretado a código nativo para su ejecución

- El **J2SE (JDK Standar Edition)** incluye bibliotecas muy completas y extensas, que permiten la implementación de cualquier tipo de aplicación:

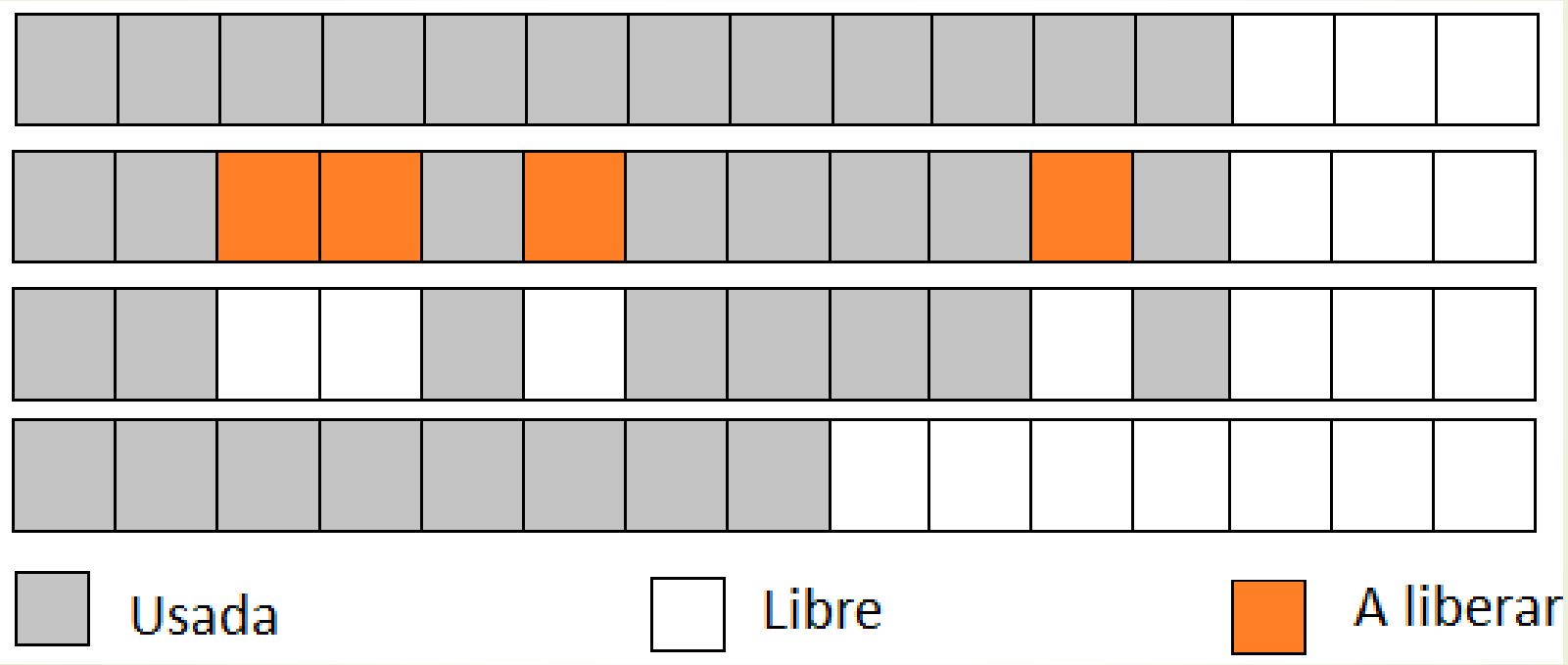
- ✓ **Seguridad**
- ✓ **Componentes (JavaBeans)**
- ✓ **Internacionalización**
- ✓ **E/S**
- ✓ **XML**
- ✓ **Redes y acceso a internet**
- ✓ **Programación distribuida**
- ✓ **Matemáticas de precisión**
- ✓ **Sonido**
- ✓ **Interfaz de usuario (AWT, SWING)**
- ✓ **Gráficos 2D.**
- ✓ **Manipulación, carga y descarga de imágenes.**
- ✓ **Impresión.**
- ✓ **Acceso a bases de datos (JDBC).**

- Java sigue un esquema no tradicional de compilación/ejecución, ya que se trata de un lenguaje **semi-interpretado**:

- Para su ejecución se requiere un JRE específico de la plataforma. El JRE está formado por una **máquina virtual java (JVM)** y las librerías de clases.

- La **JVM interpreta el bytecode** o realiza su compilación **just-in-time (JIT)** para que su ejecución sea más eficiente.

Recolector de basura de Java



- Una de las características de Java es que dispone de un **recolector de basura**.

- El entorno de ejecución de Java es el responsable de gestionar el **ciclo de vida de los objetos**.

- Así cuando **un objeto deja de usarse** por el programa, el recolector de basura de Java borra el objeto, liberando así los recursos que ocupa descargando la **memoria del computador**.

- **Otros lenguajes** como C++, obligan al programador a eliminar aquellos objetos que dejan de usarse en el programa.

- Si el programador no hace esto **de forma correcta** se puede llegar a **agotar la memoria disponible**.

- Pudiendo provocar **un colapso** tanto en la aplicación como en el sistema.

- Los **ejecutables son pequeños** porque las librerías de clases vienen proporcionadas junto a **la JVM en el JRE** de la plataforma concreta.

- La **velocidad** debido a la interpretación y/o incluso la compilación just-in-time del bytecode produce aplicaciones **más lentas**.

- El **recolector de basura** puede suponer una sobrecarga adicional para el procesador.

- Se compila la aplicación **una única vez** y los ejecutables en bytecode obtenidos son **válidos para cualquier plataforma**.

- Es muy **robusto**. La máquina virtual es capaz de **detectar y notificar** gran cantidad de errores durante la ejecución de la aplicación.

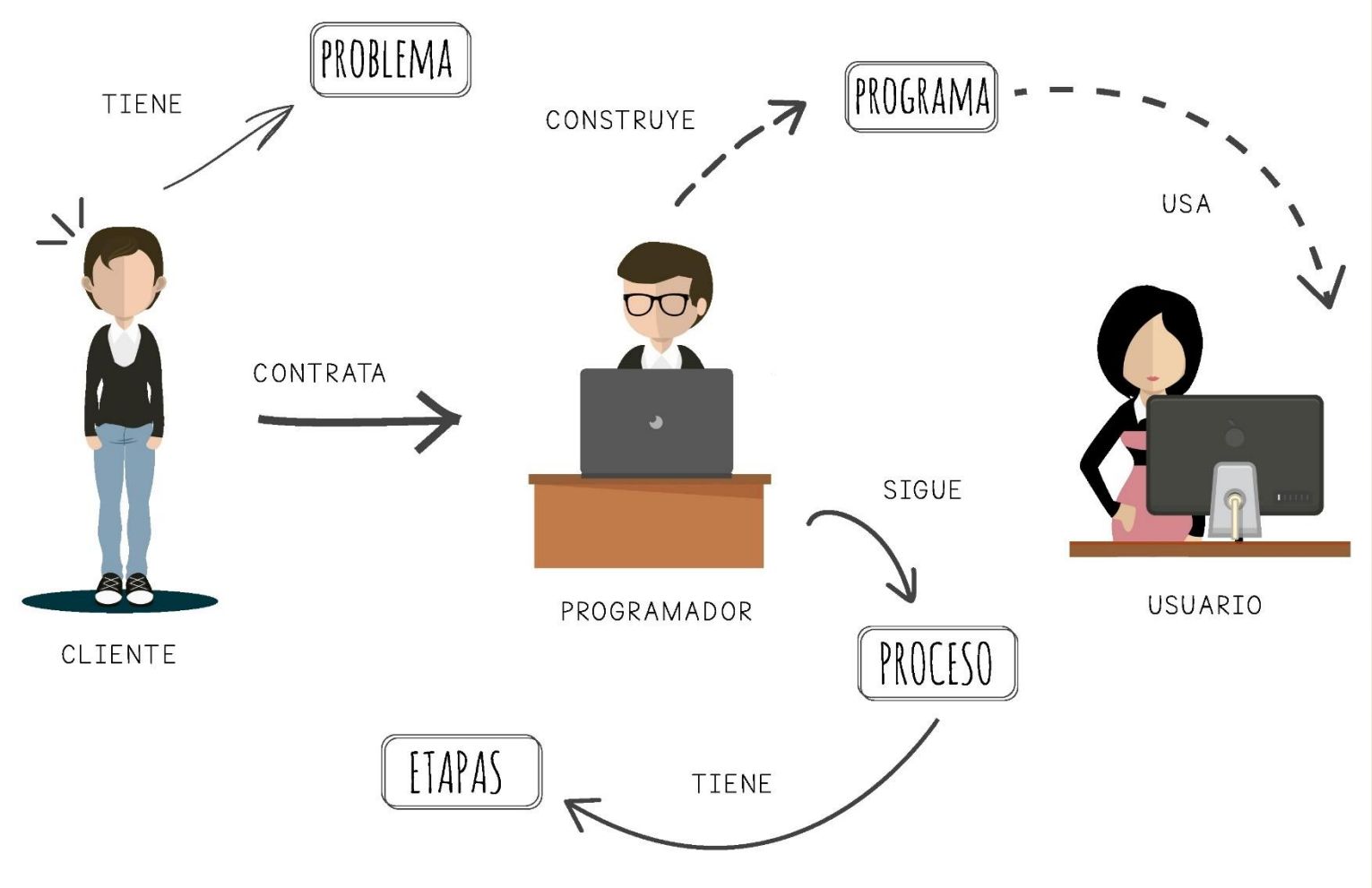
- El recolector de basura **no ocupa espacio en el ejecutable** ya que está integrado en la máquina JVM.

- La **ingeniería del software** puede definirse como la disciplina que trata los aspectos tecnológicos y metodológicos relacionados con el **diseño, producción y mantenimiento sistemático** de programas de computadoras.

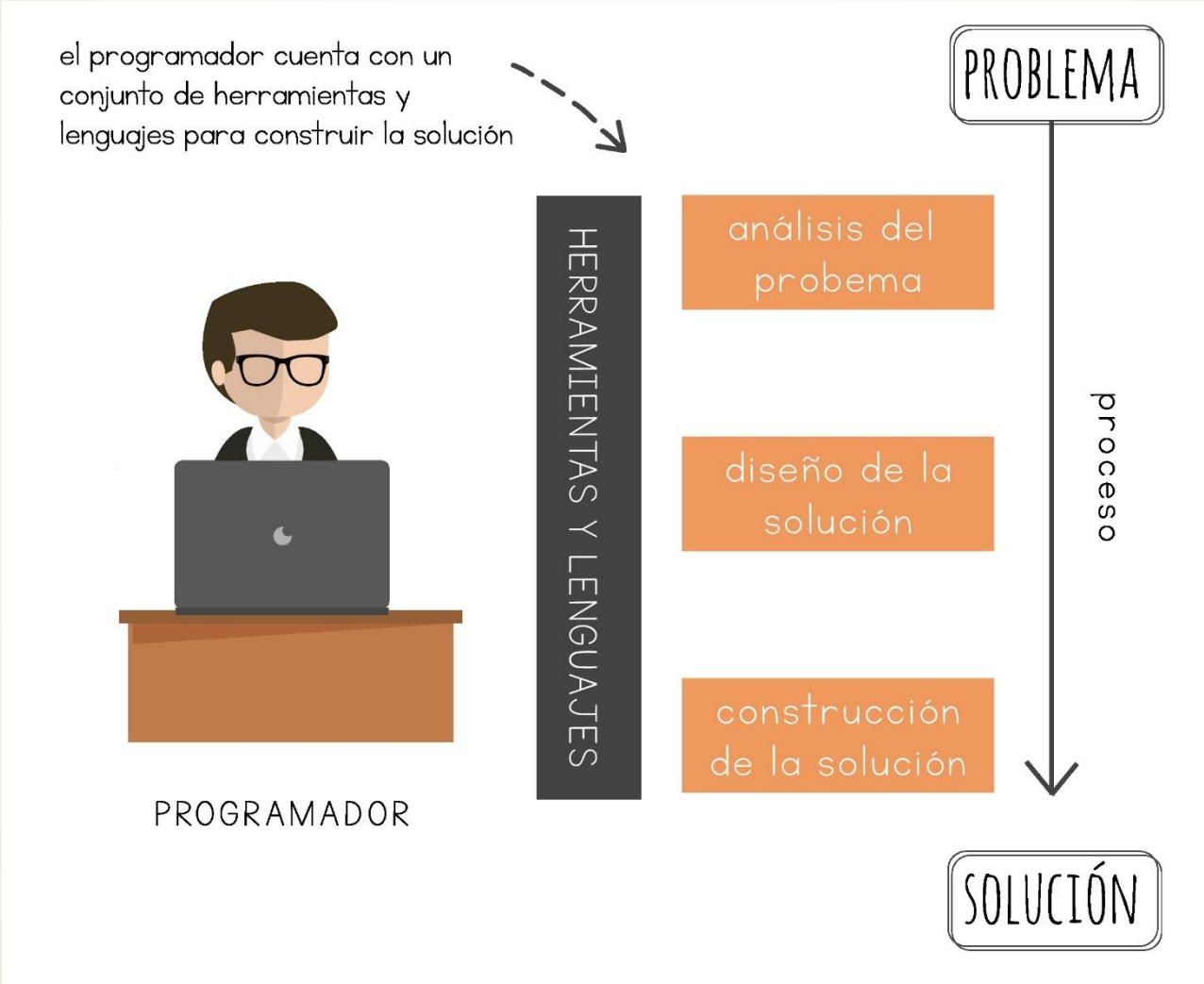
- El **proceso de desarrollo** son las diferentes **etapas** desde que se produce la necesidad de crear un software hasta que se finaliza y esta listo para ser usado por un usuario.

- No todas la aplicaciones son iguales, y por tanto su fase de desarrollo tampoco es idéntica.

Ingeniería del software



Ingeniería del software



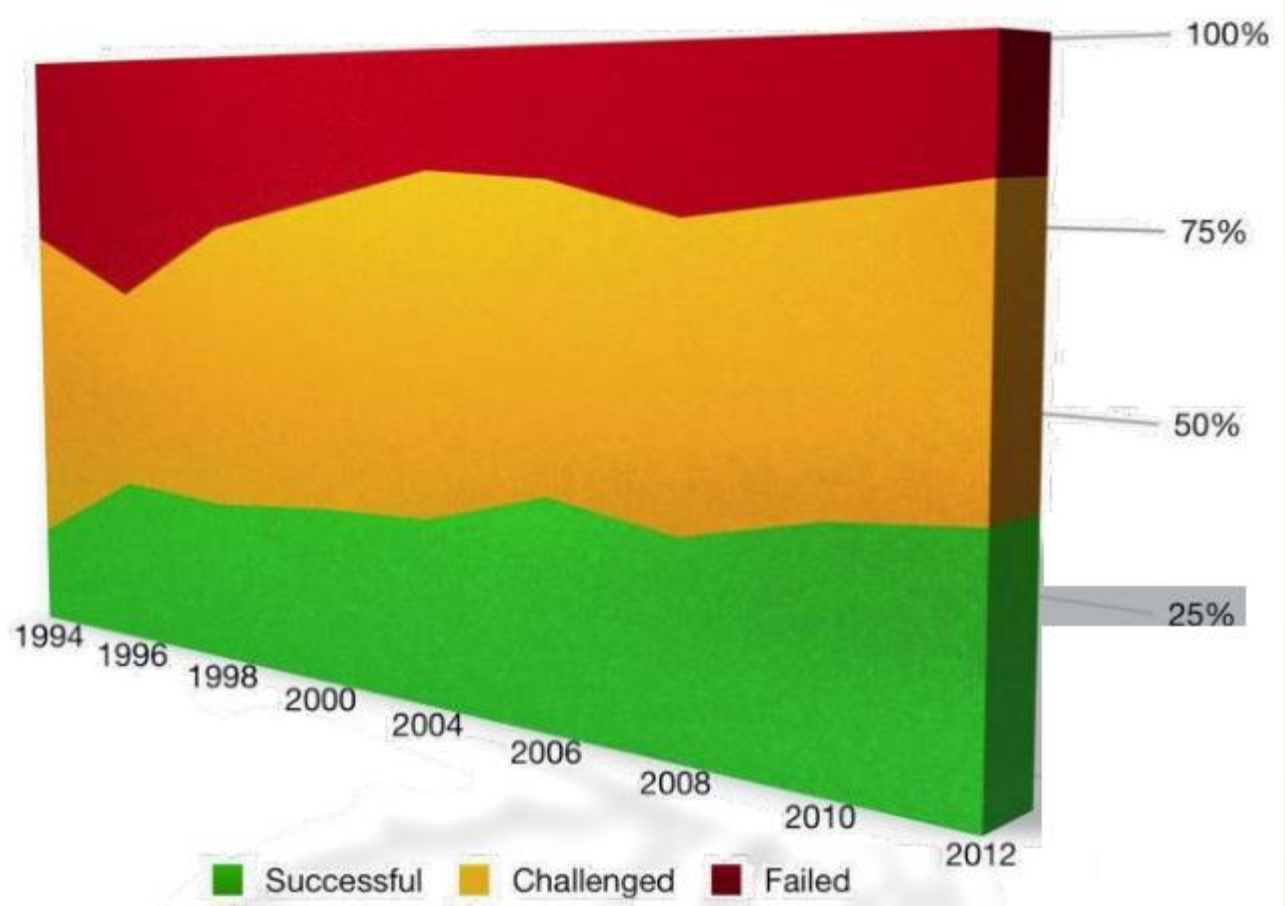
Roles en un proyecto

	Director de proyecto	Responsable de la ejecución del proyecto con capacidad ejecutiva para tomar decisiones sobre el mismo de acuerdo con el cliente.
	Ingeniero de requisitos	También denominado *analista*. Responsable de interactuar con clientes y usuarios para obtener sus necesidades y de desarrollar y gestionar los requisitos.
	Equipo de desarrollo	Conjunto de personas implicadas en el desarrollo del software: arquitecto software, diseñador de IU, programador, responsable de pruebas, administrador de BD, etc.
	Equipo de calidad	Conjunto de personas responsables de la calidad de los productos obtenidos, tanto documentación como software. Suelen ocuparse también de la calidad de los procesos.
	Cliente	Responsable de la financiación del proyecto con capacidad ejecutiva para tomar decisiones sobre el mismo. Suele tener una visión global del modelo de negocio.
	Usuario	Usuario potencial del software a desarrollar en el proyecto con una visión detallada, aunque puede que parcial, del modelo de negocio.
	Responsable TIC del Cliente	Responsable del entorno tecnológico del cliente, sobre el que se debe integrar el sistema a desarrollar.

Crisis del Software

- Muchos proyectos software comenzaron a presentar deficiencias:
  - Retraso en la entrega.
  - Falta de fiabilidad.
  - Coste excesivo.
  - Ineficiencia.
  - Mantenimiento problemática.
  - Falta de adaptabilidad y portabilidad.
  - Carencia de documentación.

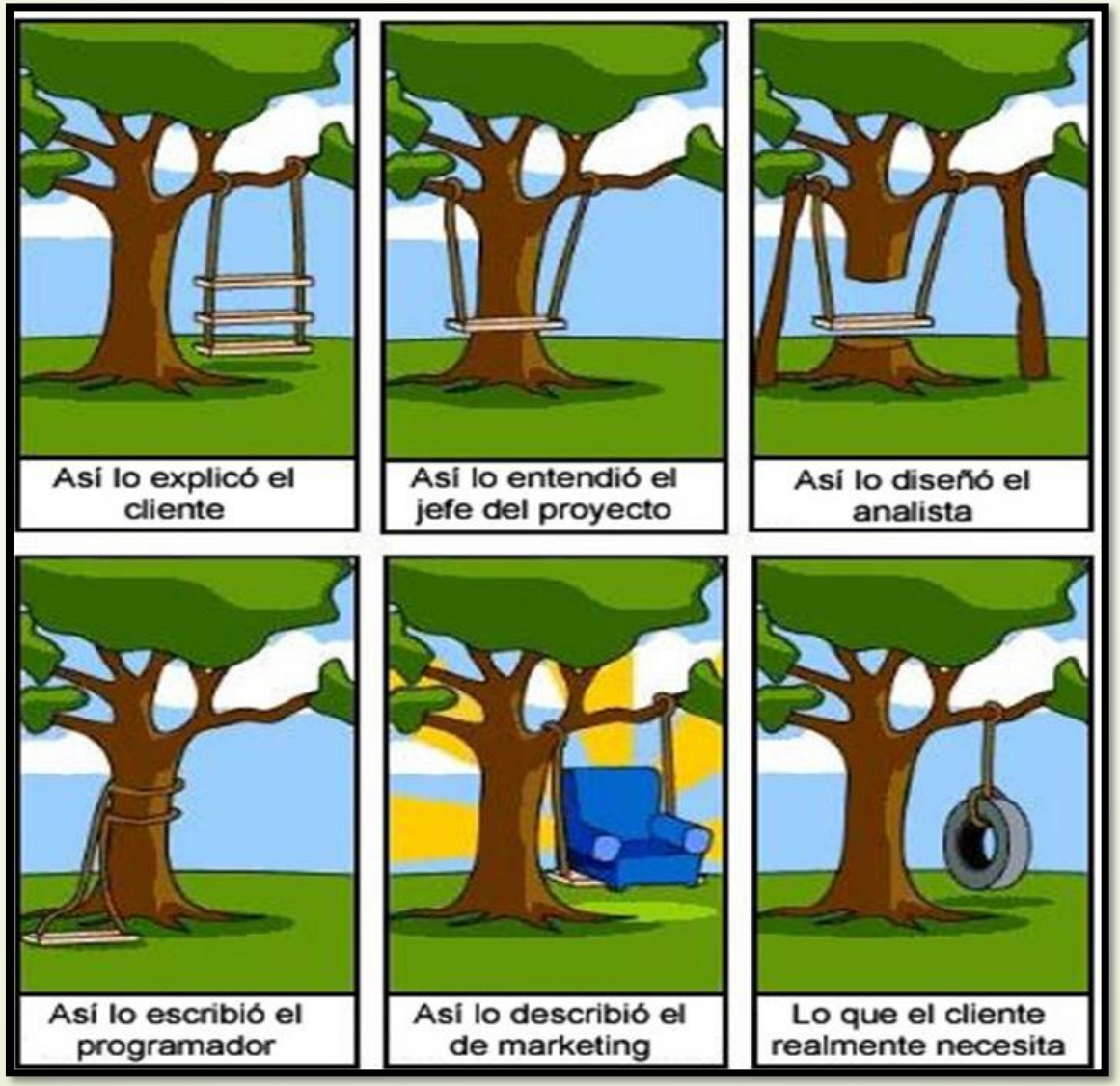
Informe CHAOS



- Según estimaciones, **el 26%** de los grandes proyectos de software **fracasan**, el **48%** deben modificarse **drásticamente** y sólo el **26%** tienen rotundo **éxito**.
- Un proyecto desarrollado de forma indisciplinada tiene muchas posibilidades de fracasar...
- Nos centramos demasiado en empezar a **programar lo antes posible** y eso es **problemático en el futuro**.
- Otro problema muy común es la **comunicación con el usuario final de aplicación**, que además raramente suele saber exactamente que quiere.

- **Factores de éxito:**
  - Expectativas realista.
  - Requisitos claros.
  - Implicación de los usuarios.
  - Visión y objetivos claros.
- **Factores de fracaso:**
  - Requisitos incompletos o cambiantes.
  - Falta de implicación de los usuarios.
  - Ya no se necesita.
  - **Falta de gestión, planificación y recursos.**

Comunicación al desarrollar software

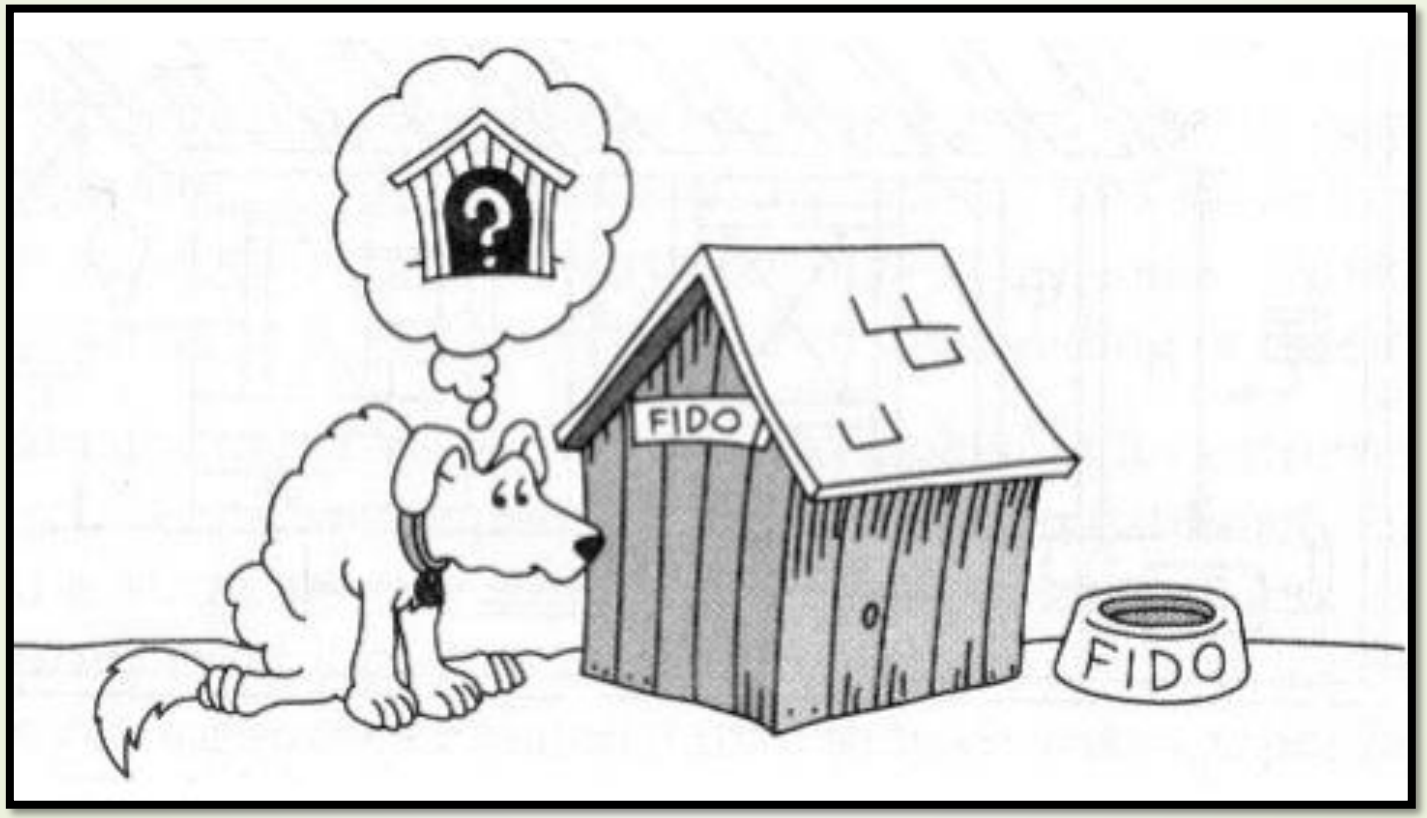


Porque pasa lo anterior

Sistemas software artesanales:

- No son complejos.
- Suelen estar construidos y mantenidos por una sola persona.
- Ciclo de vida corto.
- Pueden construirse aplicaciones alternativas en un periodo razonable de tiempo.
- No necesitan grandes esfuerzos en análisis y diseño.

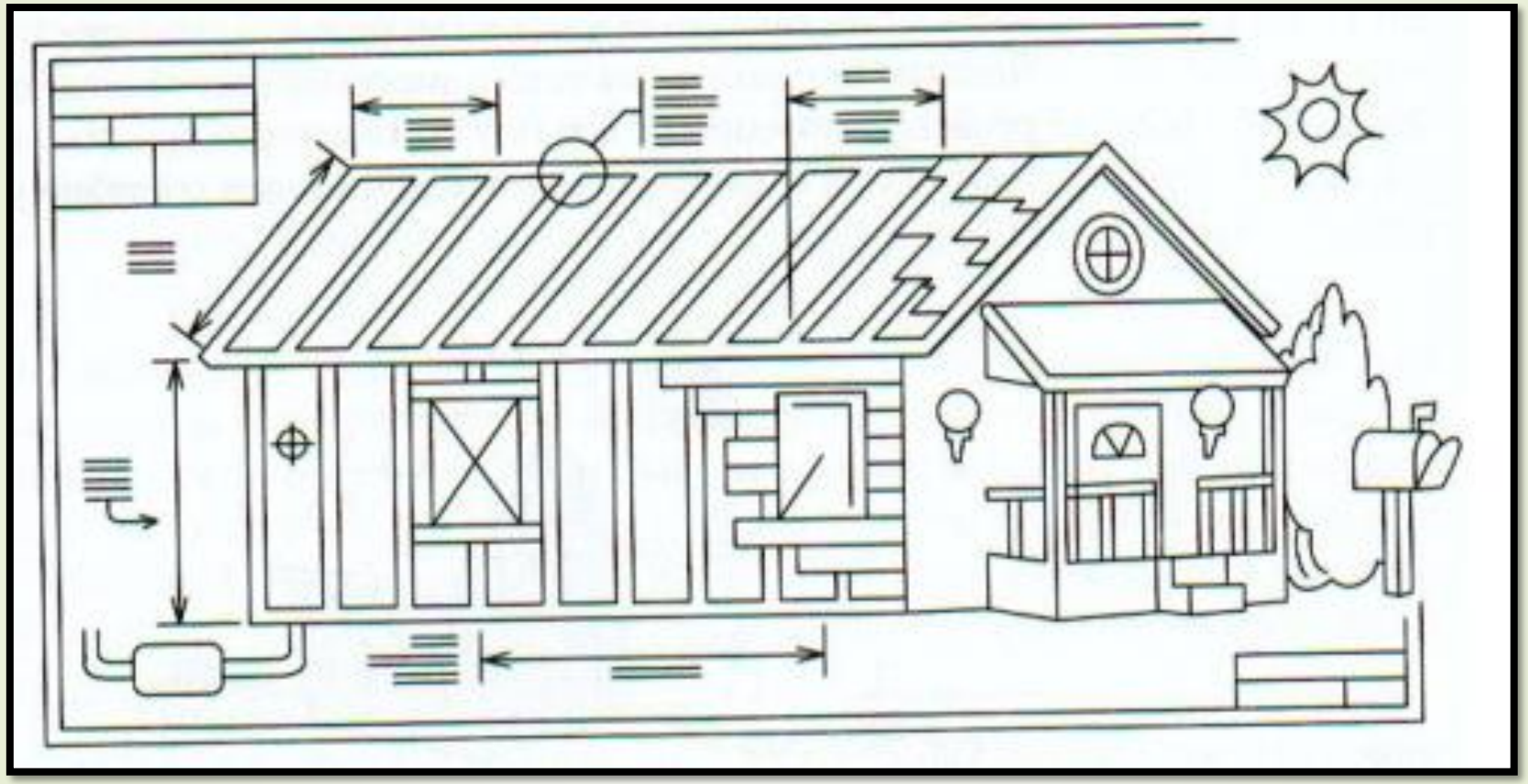
Software artesanal



Sistemas software industriales:

- Es muy difícil o imposible que un desarrollador individual pueda comprender todas las sutilidades de su diseño.
- La complejidad es una propiedad esencial de estos sistemas, que puede dominarse, pero no eliminarse.
- No puede probarse de manera exhaustiva.
- **Ejemplos:** Un sistema de reservas, anulaciones y ventas de billetes aéreos para un conjunto de compañías aéreas que se pueda utilizar en cualquier lugar del mundo.

Software industrial

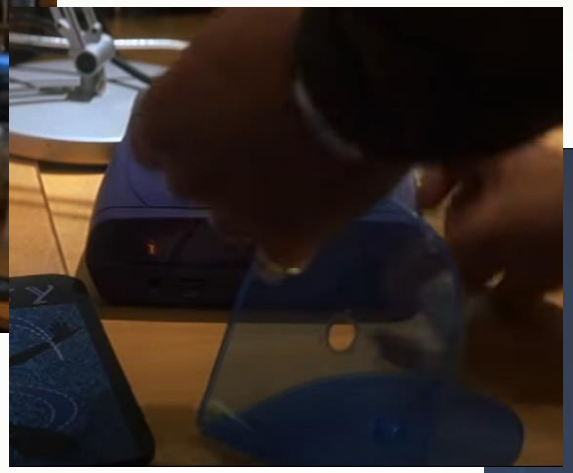


Podemos concluir que el software es complejo de forma innata:

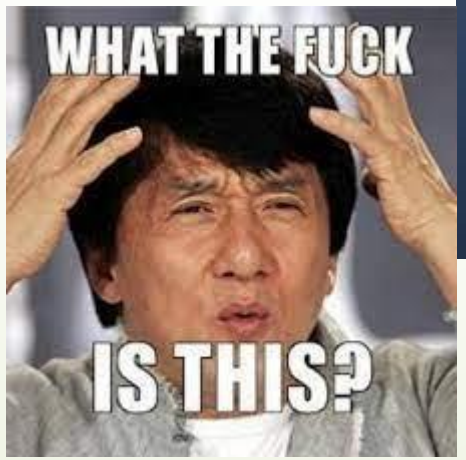
- Gran cantidad de requisitos que compiten entre sí, incluso contradiciéndose.
- La forma habitual de especificar los requisitos son grandes cantidades de texto con unos pocos dibujos.
- Los usuarios suelen tener ideas vagas de lo que desean traduciéndose en dificultades de comunicación, distintas perspectivas de la naturaleza del problema.

- Modificación de los requisitos con el paso del tiempo, pues los usuarios y desarrolladores comienzan a compenetrarse mejor.
- Dirigir un equipo de desarrolladores manteniendo la unidad de acción, conservando la integridad del diseño y manejando gran cantidad de código.
- Desarrollo rápido de prototipos (RAD).
- Control de versiones.
- Reutilización de código. No inventar la rueda una y otra vez.

Escena de la película Antitrust



```
if (buf[0] == (byte)'G' &&
    buf[1] == (byte)'E' &&
    buf[2] == (byte)'T' &&
    buf[3] == (byte)' ') {
    doingGet = true;
    index = 4;
} else if (buf[0] == (byte)'H' &&
        buf[1] == (byte)'E' &&
        buf[2] == (byte)'A' &&
        buf[3] == (byte)'D' &&
        buf[4] == (byte)' ') {
    doingGet = false;
    index = 5;
} else {
    ps.print("HTTP/1.0 " + HTTP_BAD_METHOD +
            " unsupported method type: ");
    ps.write(buf, 0, 5);
```



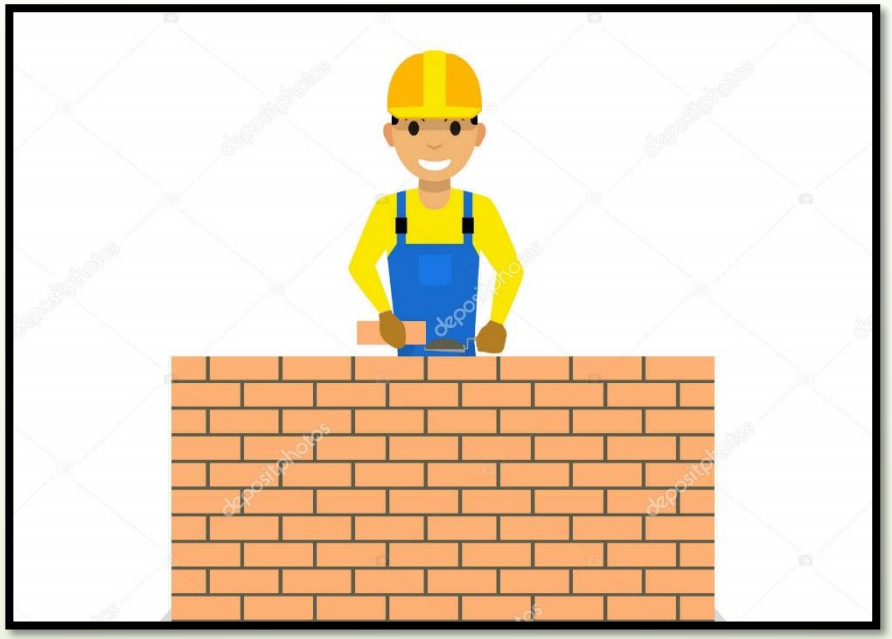
Planificación y gestión

- Desarrollar software es como construir un edificio, hay mucho que hace antes del “verdadero” trabajo:
  - Plan de obra minucioso.
  - Elegir materiales.
  - Establecer y respetar una temporización.
  - Inspeccionar frecuentemente la obra.
  - Si se comete un error hay que dedicar mucho tiempo a resolverlo.
  - La dificultad depende del tamaño del proyecto.

El coste del tiempo

- Nos referimos al coste de encontrar un error en un proyecto y tener que solventarlo.
- También al coste de introducir un cambio en el proyecto en alguna de las etapas de su desarrollo.
- Todos ellos son imprevistos y requieren de tiempo para que el proyecto de adapte y el tiempo es oro.
- Es el coste más difícil de prever y no tiene fácil solución, casi es obligado incluir en la planificación que habrá cambios y errores.

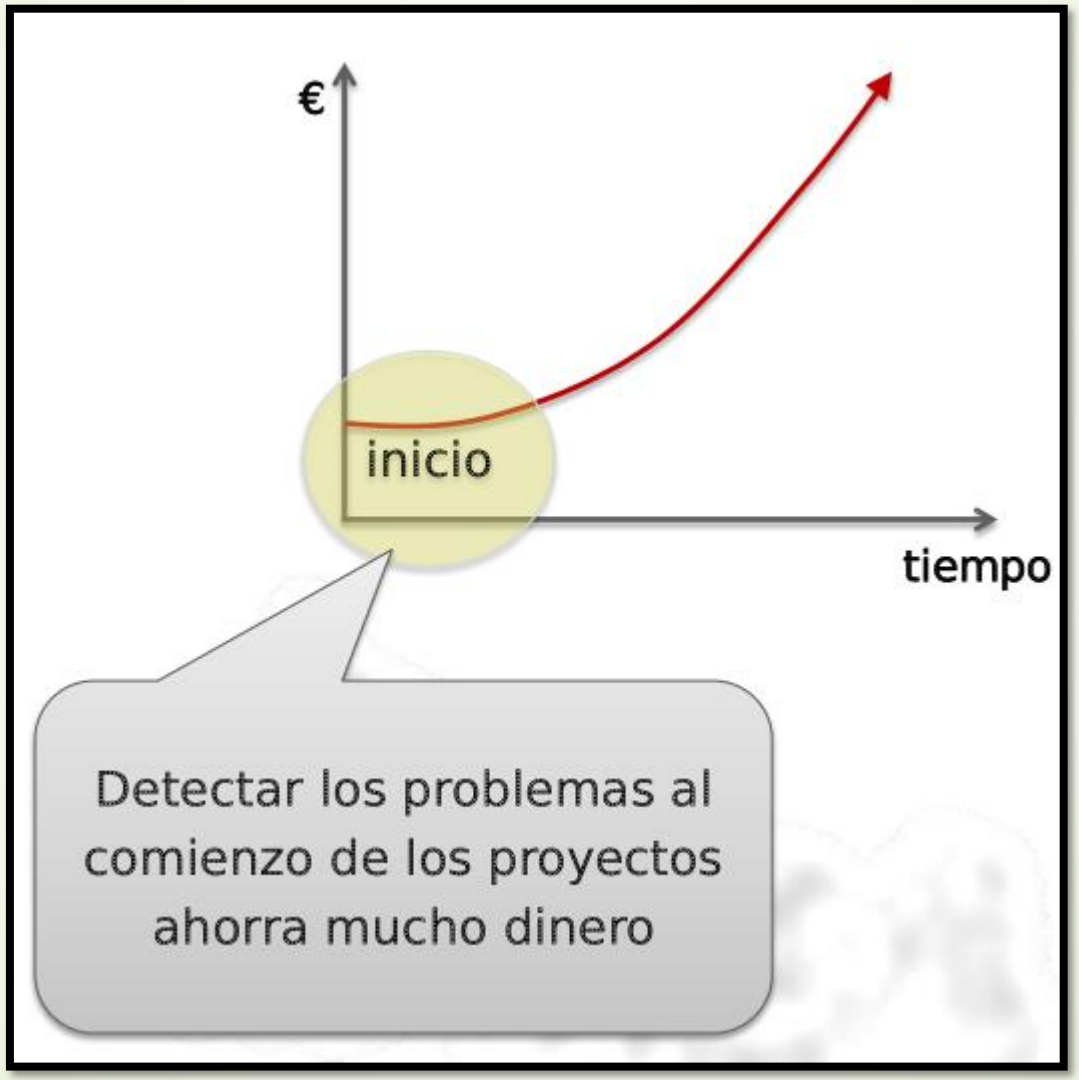
- Una de las peculiaridades del desarrollo software es que añadir más programadores no permite adelantar trabajo, más bien lo retrasa.
- La programación es una tarea de perfil muy técnico, poner al día a un grupo de programadores conlleva más tiempo todavía.



Mythical man month

- Debido al tiempo requerido por los nuevos programadores para aprender acerca del proyecto, como al aumento en la sobrecarga de comunicaciones.
- Cuando N personas tienen que comunicarse entre ellos (sin jerarquías), al aumentar N, la cantidad de comunicación M disminuye y puede incluso resultar negativa.
- Fórmula de la intercomunicación grupal: $n(n - 1) / 2$
- Ejemplo: 50 programadores generan $50 \cdot (50 - 1) / 2 = 1225$ canales de comunicación.

Análisis del coste temporal



Explicación del coste temporal

Los errores se propagan...

Problema mal planteado

↓

Especificación incorrecta
del problema mal planteado

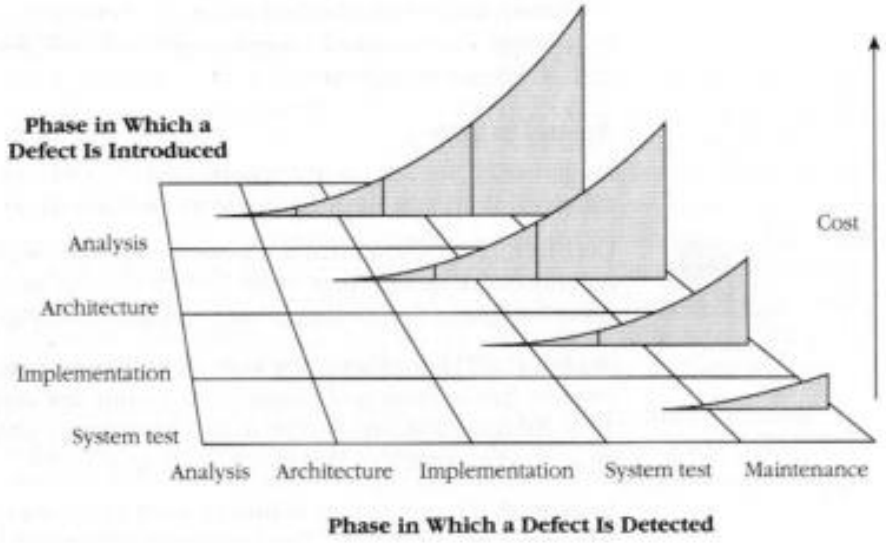
↓

Diseño inadecuado de la
especificación incorrecta del
problema mal planteado

↓

Implementación errónea del
diseño inadecuado de ...

El coste de arrastrar los errores...



Definir bien el problema

- Carrera espacial, años 60:

“Los astronautas no pueden tomar notas con los bolígrafos en ausencia de gravedad”

- Solución NASA:

“Bolígrafo espacial de avanzado diseño y gran coste”

- Solución Agencia Rusa:

“Usar lápices...”

Afrontar bien el problema

- **Problema:**

  Ordenar los números de la lotería primitiva

- **Diseño:**

  Almacenamos los números extraídos en la memoria del ordenador y usamos el algoritmo **Quicksort**, unos de los algoritmos más **poderosos** de ordenación a la vez de ser de los más **complejos.**

- **La solución es más complicada que el problema...**

Afrontar bien el problema

Como definir bien un problema

- Describe en detalle qué hace el sistema.
- No describe cómo se hace.
- Debe se correcta.
- Debe ser completa (contempla todos los casos).
- Emplea diagramas y notaciones formales.
- Debe acomodar cambios (se producirán).

Como diseñar bien una solución

- Describe cómo funciona el sistema.
- Define la estructura del sistema:
  - Los componentes, su funcionalidad y las relaciones entre ellos.
- Justifica las decisiones de diseño.
- Emplea diagramas y notaciones formales.
- Independiente del lenguaje, del S.O. y la maquina.

- El conjunto de etapas en el desarrollo software responde al concepto de **ciclo de vida del software**.
- **Análisis.**
- **Diseño.**
- **Codificación ó Implementación.**
- **Pruebas o verificación.**
- **Mantenimiento**

Fase de análisis

- El primer paso consiste en definir de la forma más **exacta posible el problema** que se quiere resolver.
- Un problema bien definido requiere de una **descripción precisa** de las especificaciones de **entrada y salida** para poder conseguir una solución efectiva y a la par eficaz.
- **Ejemplo 1:** Se necesita un programa que calcule la superficie de un rectángulo

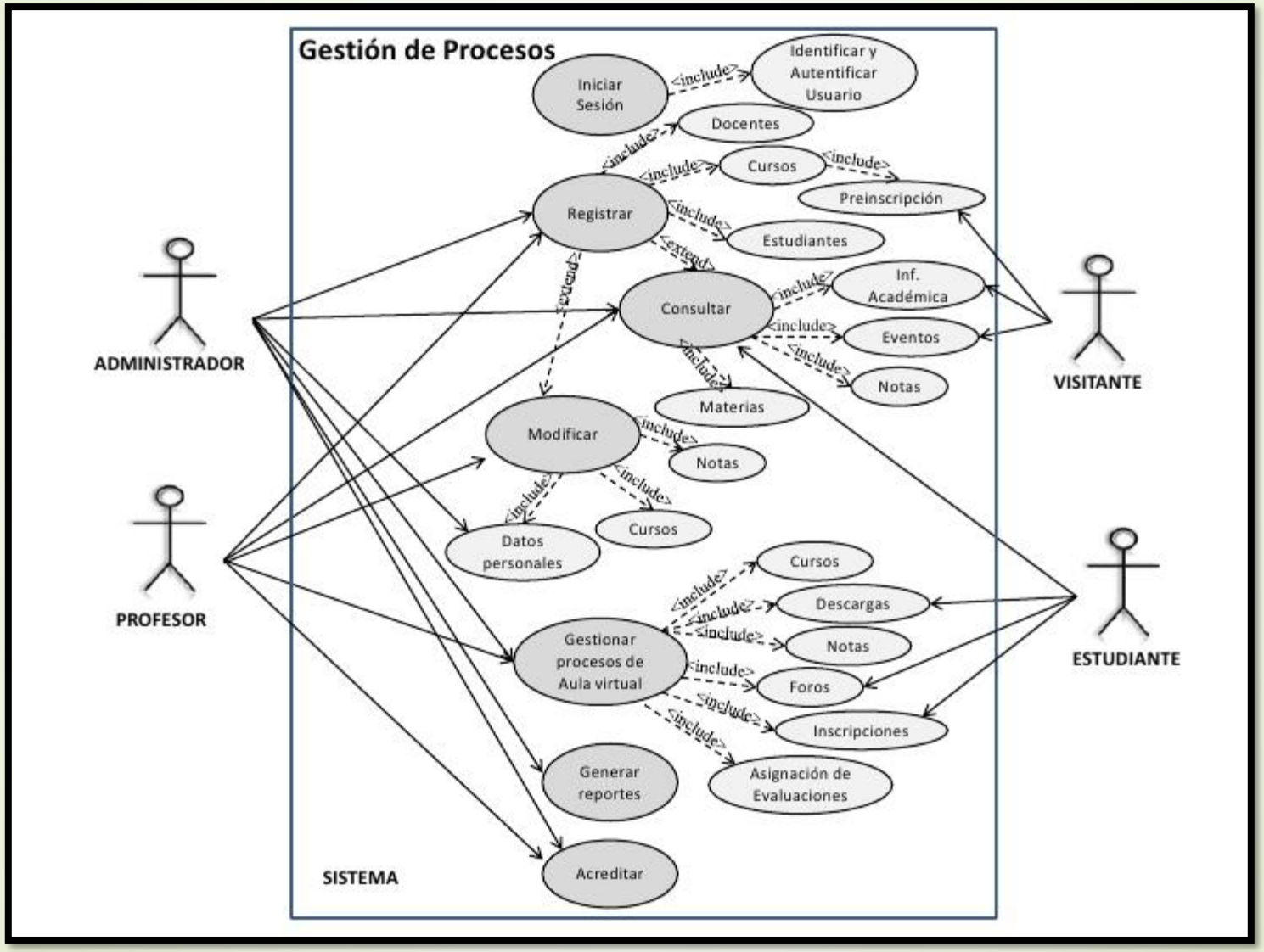
- ¿Qué datos son de entrada?
- ¿Cuantos datos se introducirán?
- ¿Cómo deben ser esos datos?
- ¿Cuales son los datos de salida?
- ¿Cuántos datos de salida se producirán?
- ¿Qué precisión tendrán los resultados?
- ¿Cómo se mostrarán?

- Estas preguntas surgen de hablar con el **cliente-usuario** de la aplicación que nos contara la **idea global** de lo que quiere.

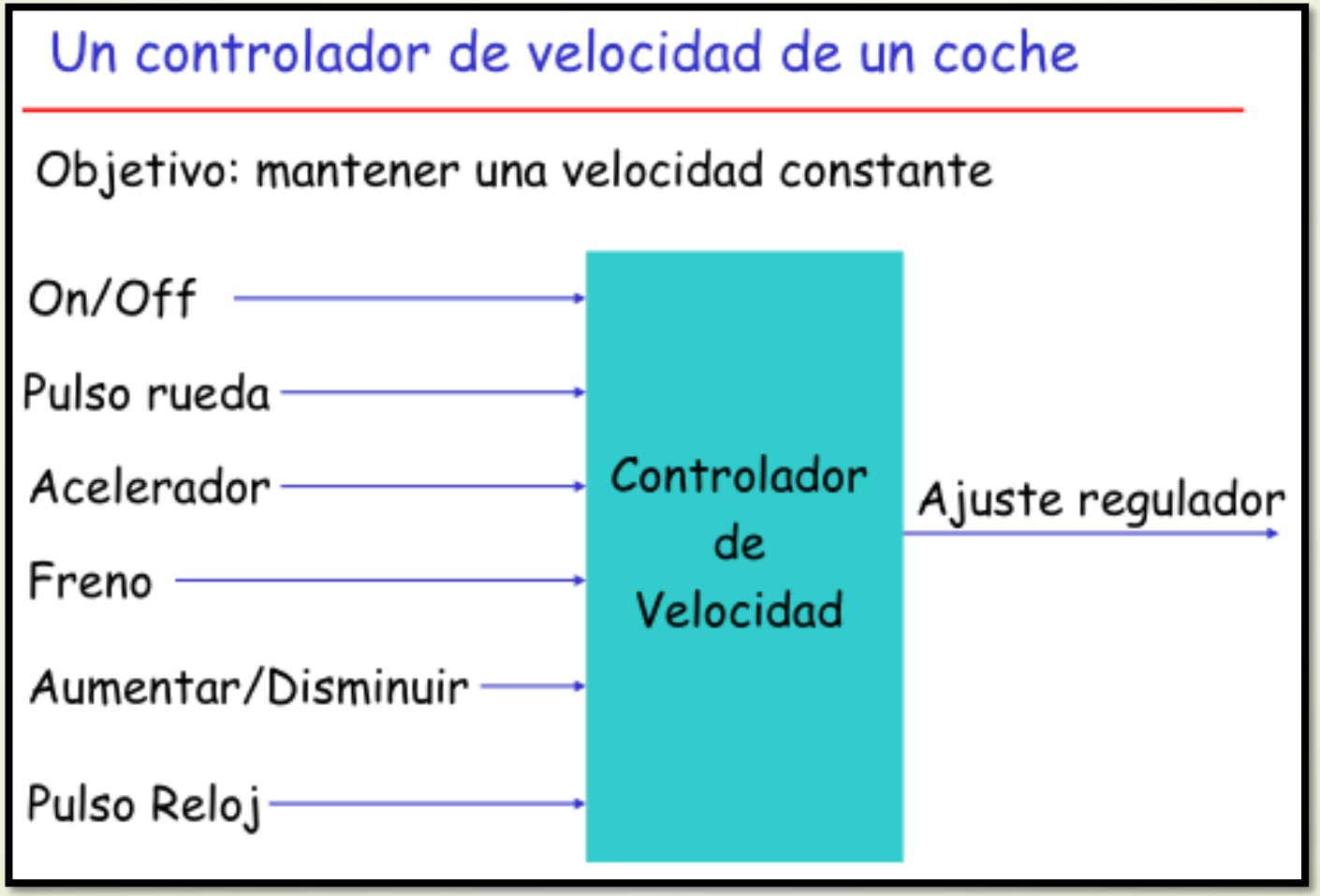
- Aunque el cliente crea que sabe lo que el software tiene que hacer es necesario una **buena habilidad y experiencia para reconocer** requisitos incompletos, contradictorios o incluso innecesarios.

- Se suelen elaborar **informes y/o diagramas** para recopilar todo lo obtenido en las entrevistas.

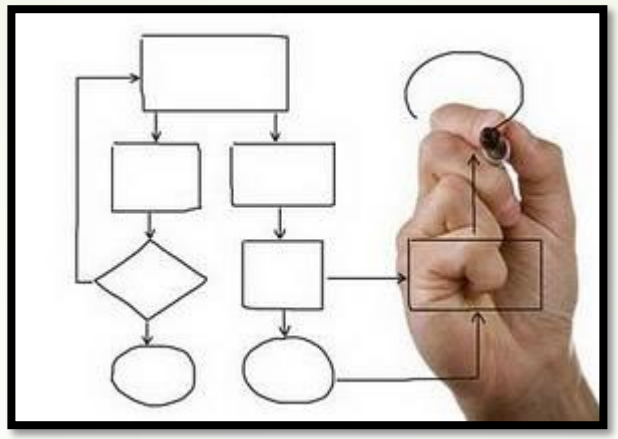
Diagramas de análisis



Ejemplo sencillo



Fase de diseño



- El siguiente paso consiste en **diseñar una solución** al problema planteado, **independientemente del lenguaje** de programación a usar.

- A la hora de enfrentarse al diseño de una solución existen un enfoque principal llamado diseño **descendente o top-down**.

- El diseño descendente se formula un resumen del sistema, **sin especificar detalles.**

- Después cada parte del sistema **se refina diseñando con mayor detalle**, **hasta** que la especificación completa es lo suficientemente **detallada para ser construida.**

- Se basa en la idea del "**divide y vencerás**" que consiste en dividir el problema en subproblemas **más fáciles que resolver** que el principal.

- Para ello se comienza por **identificar** las tareas **más importantes** a ser ejecutadas.

- Esto sirve como una **primera descripción del algoritmo** y proporciona una primera lista de actividades a ejecutar.

- Normalmente la **mayoría** de los pasos son **incompletos** y deben ser refinados, añadiendo más detalles a estos pasos y así obtener una nueva lista de actividades.

- Esto se **repite indefinidamente** mientras se considere necesario.

- **Ejemplo 1:** Se necesita una aplicación que calcule la superficie de un rectángulo.

- Este problema se puede dividir en 3 subproblemas:

  1. **Entrada de datos.**
  2. **Cálculo de la superficie.**
  3. **Salida de los resultados.**

- De este modo el problema ha quedado reducido a **3 subproblemas más simples**.

- La **solución de cada uno** de ellos nos dará **la solución del problema principal**.

¿Se puede refinar aún más cada paso?

- **Subproblema1:** Entrada de datos

  1.1 Pedir la altura del rectángulo

  1.2 Pedir la base del rectángulo

- **Subproblema2:** Calculo de la superficie

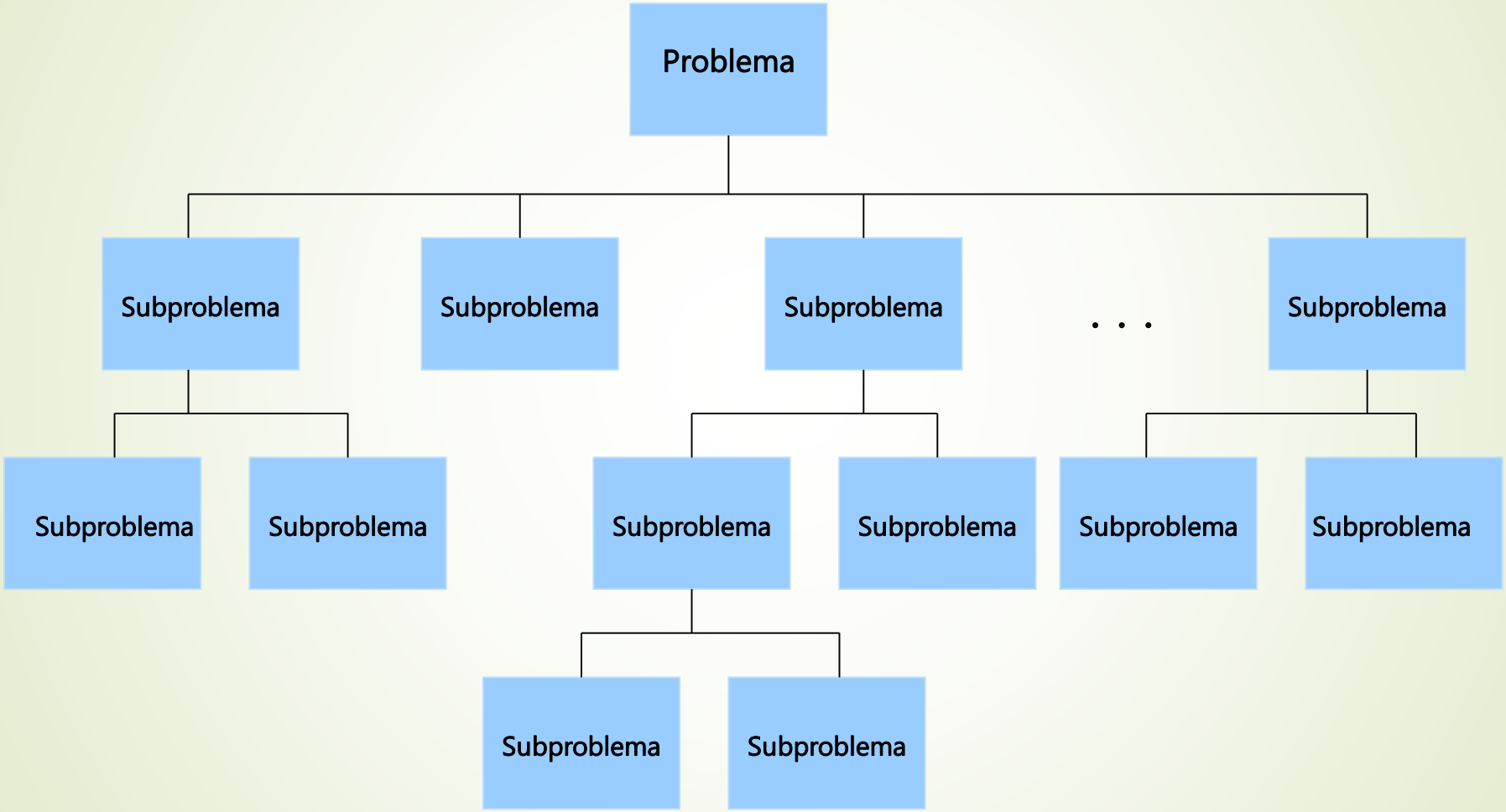
  2.1 superficie = base * altura

- **Subproblema3:** Salida de resultados

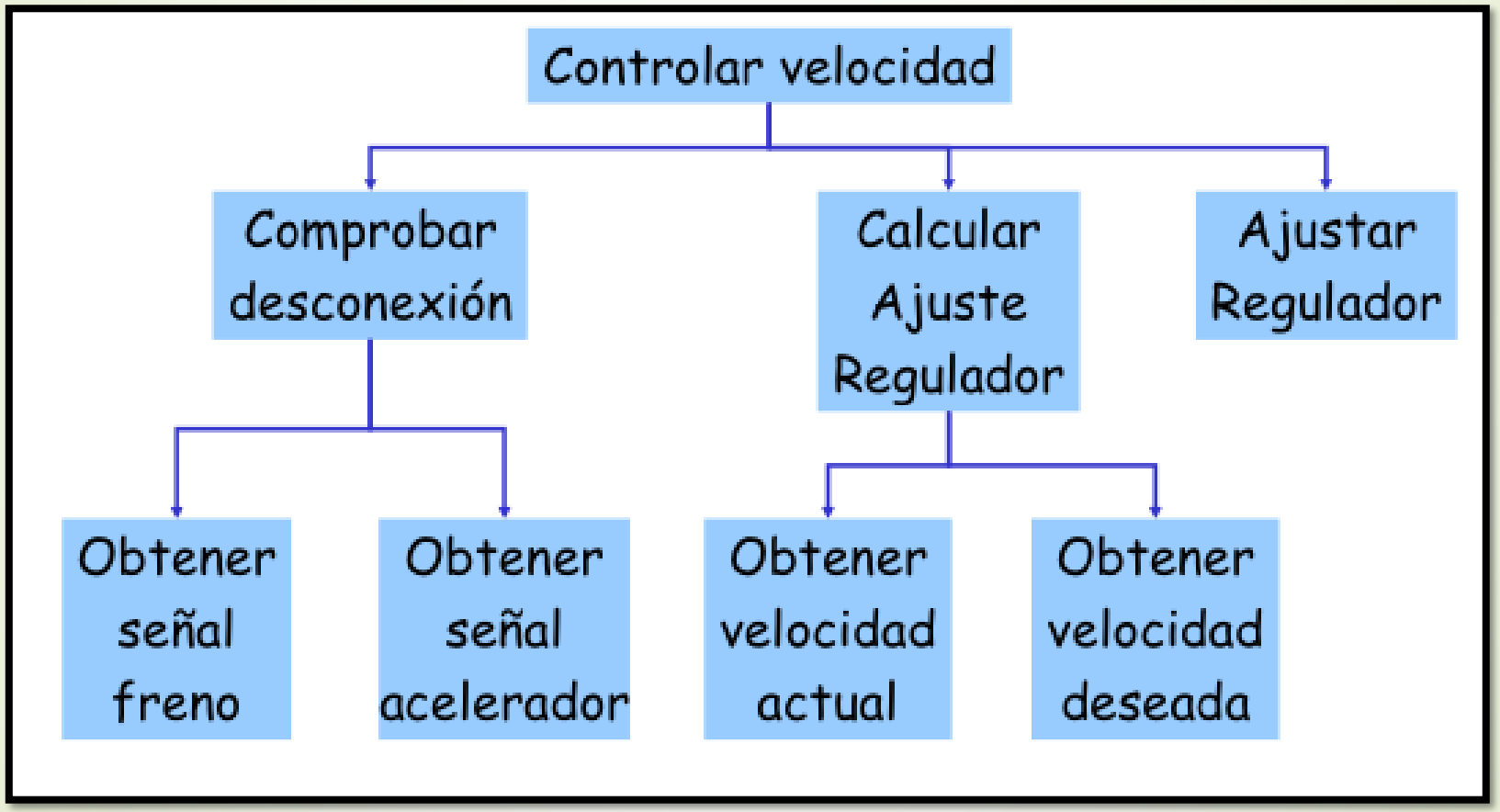
  3.1 Mostrar la base y la altura introducidas por el usuario

  3.2 Mostrar la superficie calculada

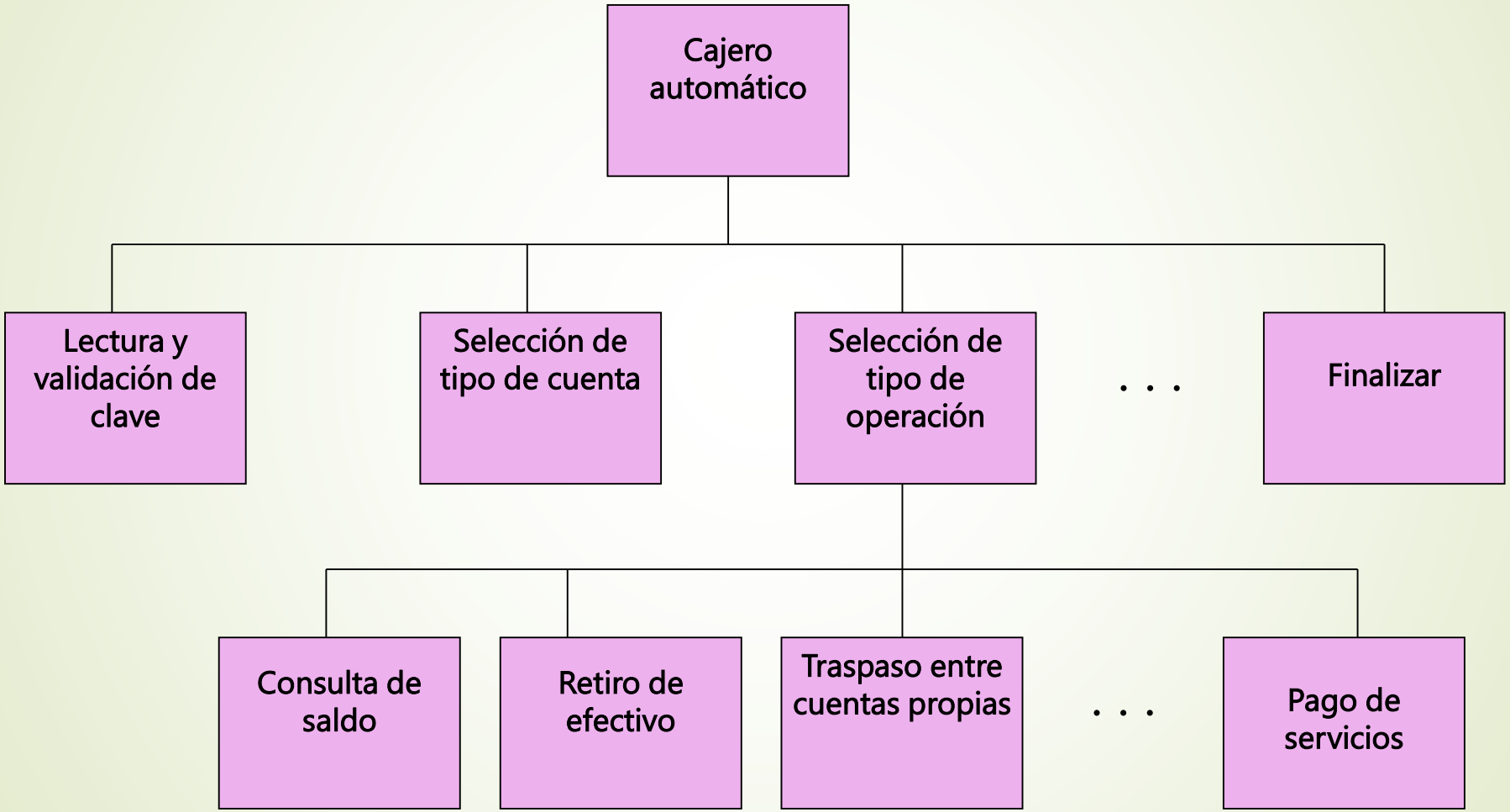
Idea del diseño descendente



Ejemplo controlador de velocidad



Ejemplo diseño descendente

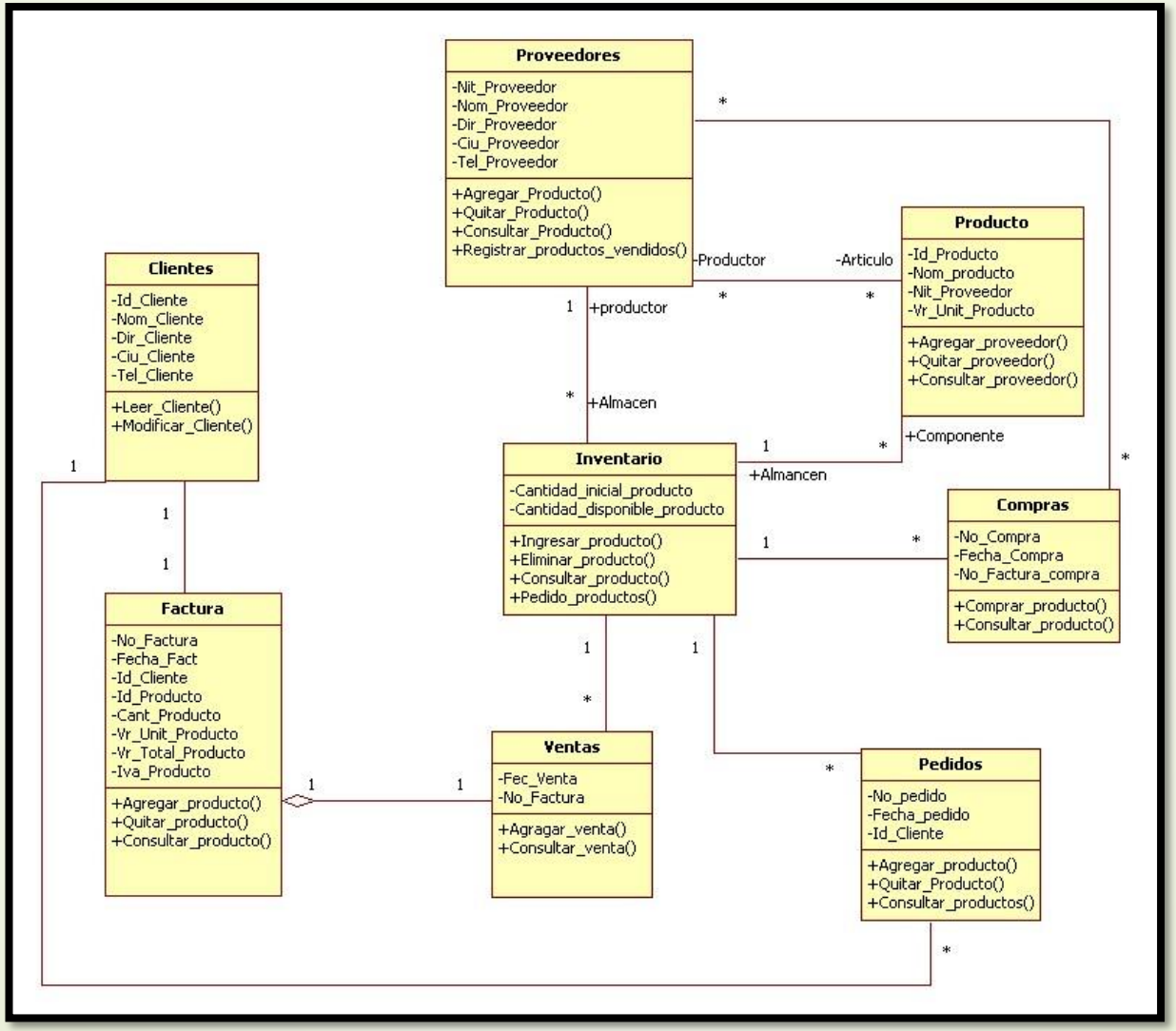


- Lo que hemos visto es un enfoque básico de la fase de diseño desde el punto de vista de los algoritmos, o sea, enfocado en la acción.

- Existen más técnicas de diseño además del diseño descendente más enfocada a los elementos que forman parte de un sistema.

- Dicho enfoque es el diseño orientado a objetos que se tratan con los diagramas de clases.

Ejemplo diagrama de clases



Fase de implementación

- El **diseño realizado se traducirá** posteriormente a un lenguaje comprensible por la computadora.

- Gracias a las etapas anteriores, el programador contará con un **análisis completo del sistema** y una especificación de la estructura básica que se necesitara para codificarlo.

- **Esto no siempre es así**, las dificultades a la hora de concretar el código son recurrentes y suele ser **necesario modificar el diseño**.

Fase de pruebas o verificación

- A partir del código fuente se generará el programa ejecutable y se verifica que funciona **correctamente**, que no se comporta de **forma anómala** y que se **adecua a lo pedido**.

- Las pruebas la realiza, idílicamente, **personal diferente al que codifico la aplicación**, con experiencia en programación, personas capaces de **saber en que condiciones un software puede fallar de antemano** sin un análisis previo.

Fase de mantenimiento

- Es la modificación de un producto de software después de la entrega, para corregir errores, mejorar el rendimiento, u otros atributos.
- El mantenimiento del software es una de las actividades más comunes en la ingeniería de software
- A finales de los años 1970, un famoso y ampliamente citado estudio de encuesta por Gabriela castillo, expuso la muy alta fracción del coste del ciclo de vida que estaban siendo gastados en mantenimiento.
- Esto indica que en fases anteriores no se han hecho cosas bien

Ciclos de vida del software

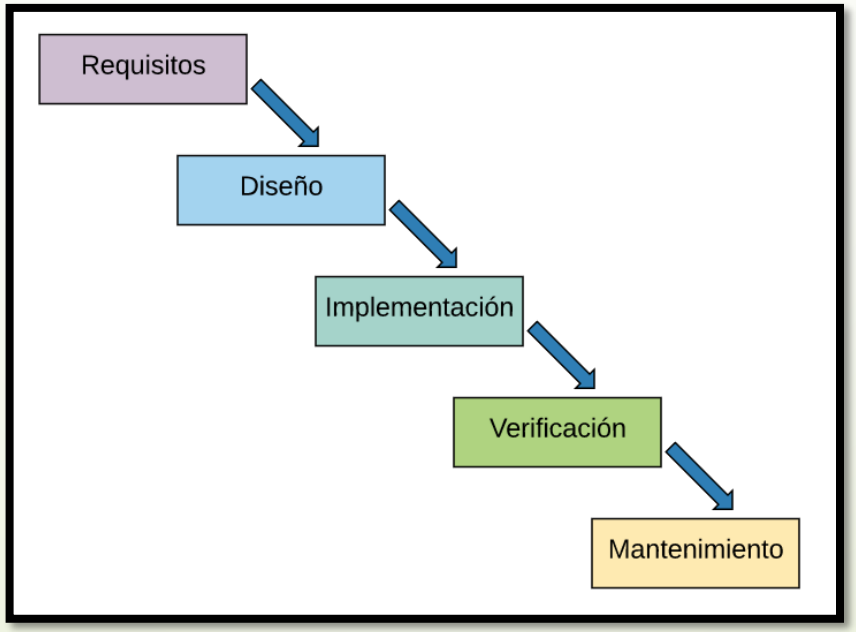
- El **ciclo de vida** del software establece **el orden** de las etapas y **duración** de las mismas.

- Diversos autores han planteado distintos modelos de ciclos de vida, pero los más conocidos y utilizados son los que aparecen a continuación:

  - ✓ **Modelo en cascada.**
  - ✓ **Modelo iterativo.**

Modelo en cascada

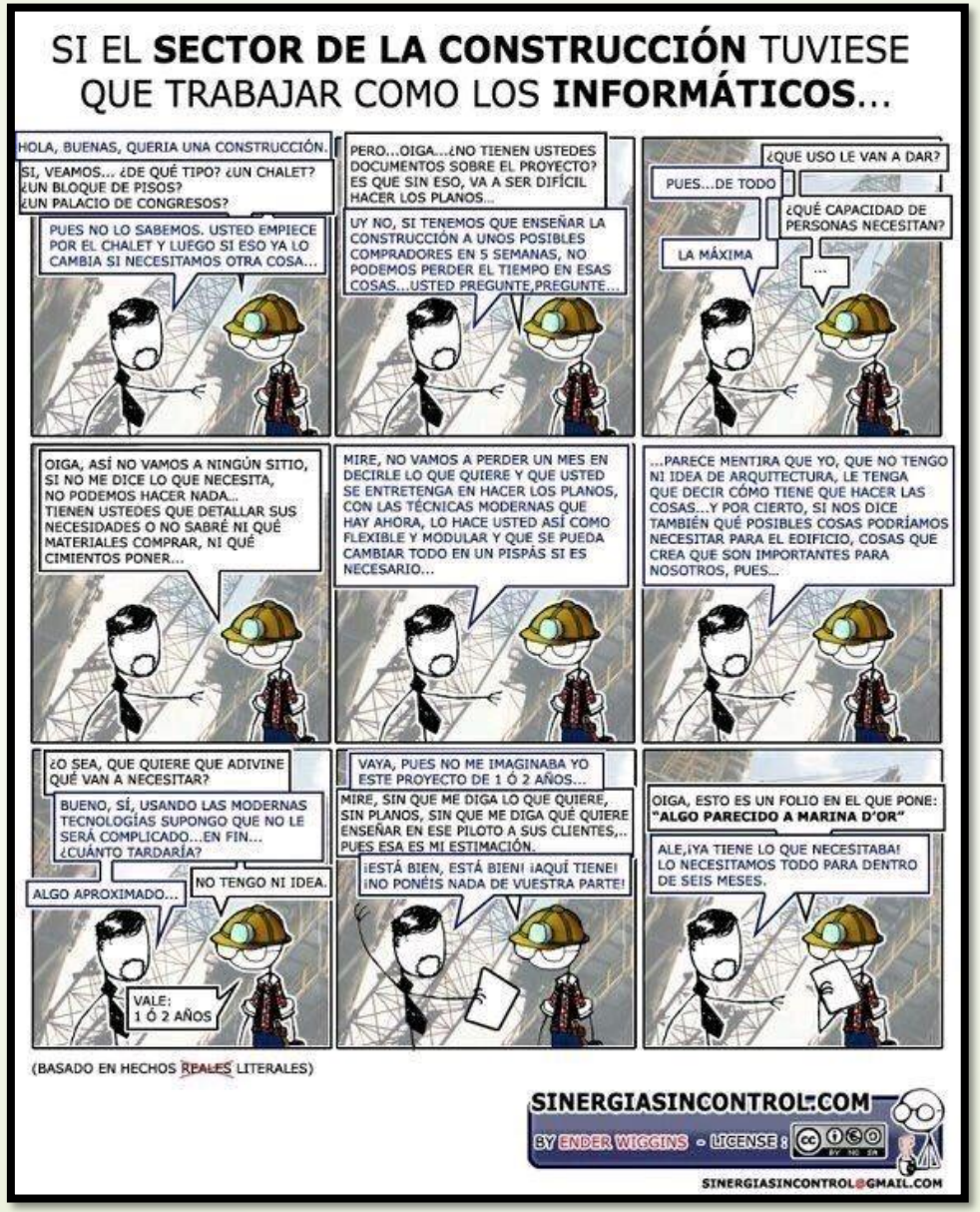
- Es el **modelo clásico** donde cada etapa **es rígida**, esto quiere decir que una vez se ha avanzado una etapa **es imposible volver atrás** y cualquier fallo tendrá que resolverse en la próxima etapa.



No siempre es posible

- Un modelo de producción en cadena industrial es muy poco probable de que funcione con el software porque no es fácil obtener una visión inicial y final clara de lo que se quiere conseguir.
- Piensa en construir un avión comercial Boieng 787.
- Piensa en construir un software de gestión empresarial.
- Pensando poco tiempo, ¿De cuál tienes ahora mismo una visión más clara?
- Si el avion vuela ya es viable ¿no? **MVP**
- ¿La gestión de una empresa esta siempre satisfecha?

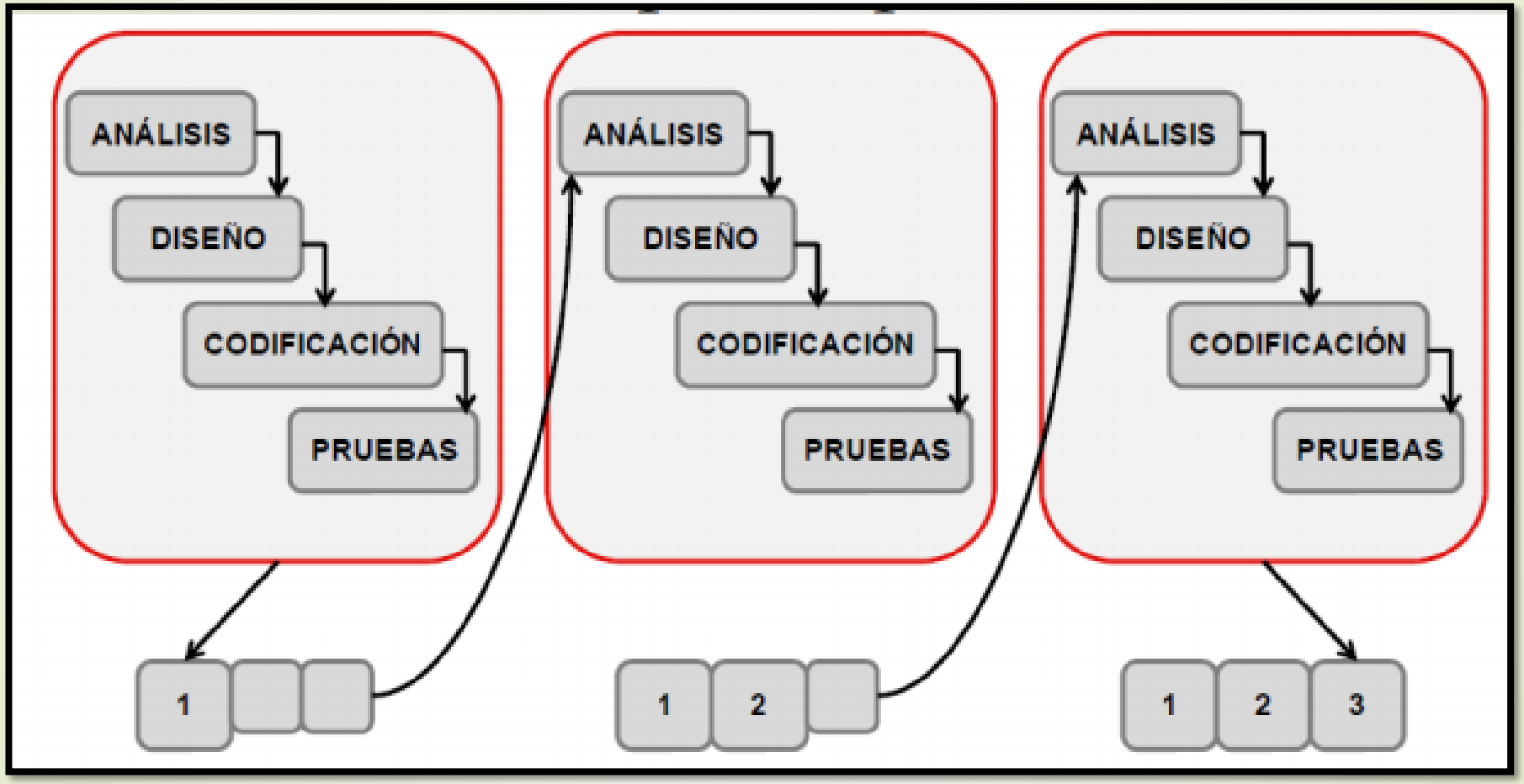
No siempre es posible



Modelo iterativo

- Se considera un **modelo evolutivo**, es decir, el software no es para nada algo rígido, **va cambiando en todas sus fases** de desarrollo y el objetivo final nunca esta claro.
- Esta metodología se basa en la idea de desarrollar una **implementación inicial**, **mostrárselo al cliente** y desarrollar sucesivas versiones (iteraciones) hasta obtener el sistema requerido.
- Cada que vez que se le muestra al cliente surgen requisitos nuevos que se pueden añadir y/o corregir

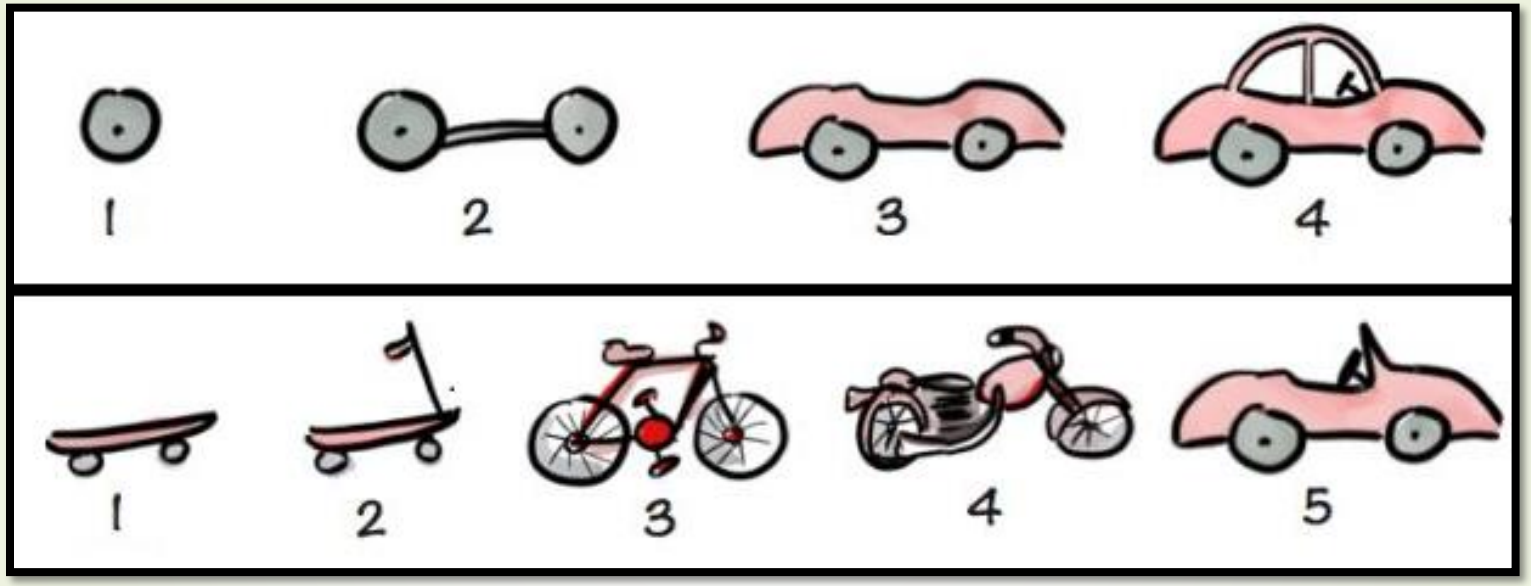
Modelo iterativo



Modelo iterativo

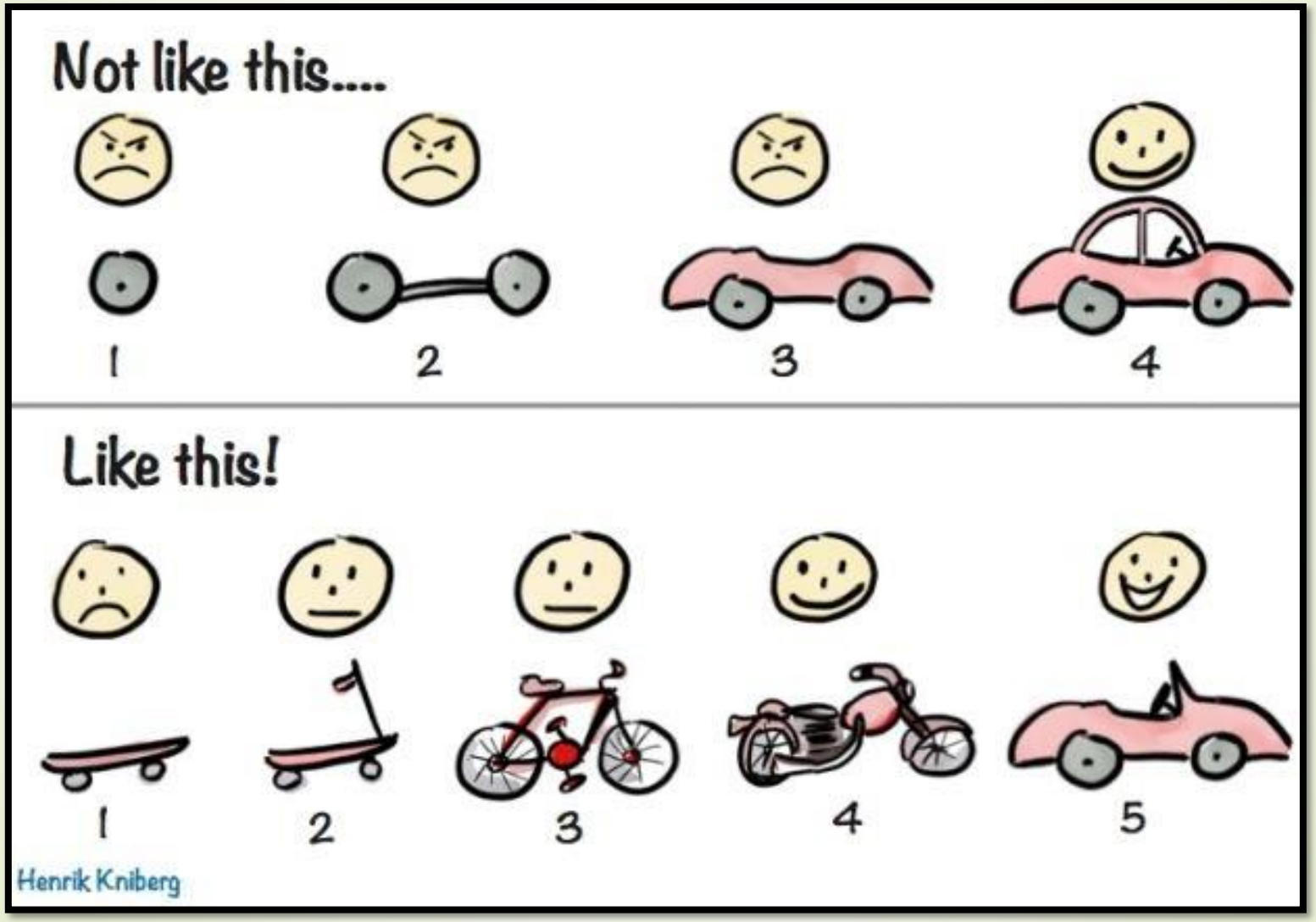
- Unos de los principales problemas del desarrollo software es los costes por perdida de tiempo a la hora de realizar cambios en un proyecto durante si ciclo de vida.
- Por ello a lo largo de los años han surgido metodologías para admitir mejor dichos cambios y se llaman metodologías agiles.
- Todo esto está muy bien pero derivan de un proceso de creación industrial que no casa con todo lo visto hasta ahora.

Una forma de entender el modelo iterativo



¿Cuál creéis que sería la forma más correcta?

Una forma de entender el modelo iterativo



Metodologías agiles

- Son aquellas que permiten adaptar la forma de trabajo a las condiciones del proyecto, consiguiendo flexibilidad e inmediatez en la respuesta para amoldar el proyecto y su desarrollo a las circunstancias específicas del entorno.
- Mejoran la satisfacción del cliente dado que se involucrará y comprometerá a lo largo de todo el proyecto
- Mejora de la motivación e implicación del equipo de desarrollo

Metodologías agiles

- Una de las máximas de su aplicación es que se trabaja a través de entregas parciales del producto.
- Gracias a las entregas parciales y a la implicación del cliente será posible eliminar cualquier característica innecesaria del producto.
- De lo anterior se deduce una mejora de la calidad del producto

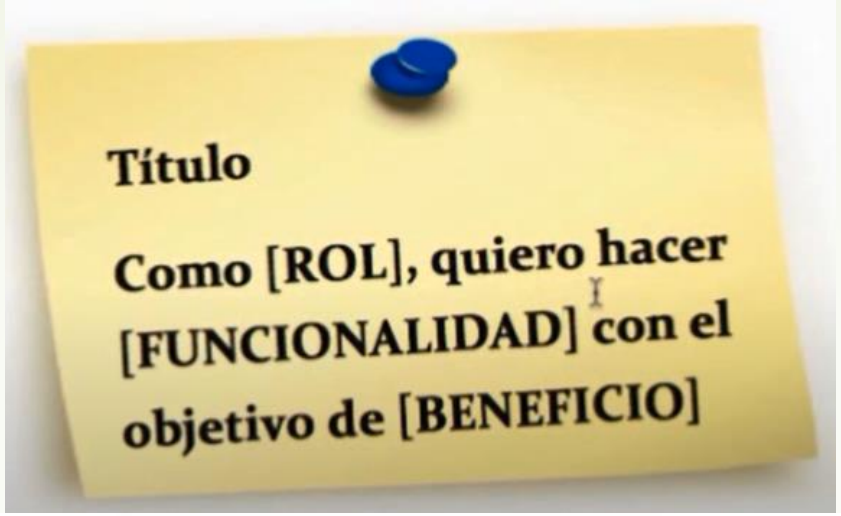
Metodologías agiles

- A través de las metodologías agiles de van creando una serie de filosofías que son adoptadas por las empresas y que forman parte del marketing en el mundo del desarrollo software.
- No es otra cosa que vender no solo el programa si no también la forma o metodología mediante la cual lo has desarrollado.

Organización y reparto del trabajo

Organización y reparto del trabajo

Historias de usuario



Historias de usuario

- Representan una funcionalidad.
- Es algo que al ser introducido aporta valor al usuario.
- Debe ser pequeña y memorizable, por lo que se suele escribir en un post-it o tarjetas.
- La tienen que poder desarrollar como mucho por dos programadores durante una semana.
- Esto se le suele llamar el metodo Kanbam.

DevOps

- Básicamente, **DevOps** consiste en eliminar las barreras entre dos equipos que anteriormente estaban aislados, el de desarrollo y el de operaciones.
- Los dos equipos colaboran para optimizar la productividad de los desarrolladores y la confiabilidad de las operaciones.
- Se esfuerzan por comunicarse con frecuencia, incrementar la eficacia y se hacen totalmente responsables de sus servicios, a menudo más allá de lo que tradicionalmente abarcarían sus funciones.

Metodologías agiles

- Como hemos podido comprobar, llevar a cabo todas estas tarea no es solo ser o no buenos programadores, ser o no buenos organizadores del trabajo.
- Conlleva tener una gran fuerza de voluntad una implicación bestial en el proyecto a desarrollar.
- Lo que lleva a **factores emocionales** y a la necesidad de generar una **identidad de grupo**.
- Por eso todo lo que se ve en series como Silicon Valley ó similares.

Bibliografía

- **Braude, E.J.: "Ingeniería del Software: una perspectiva orientada a objetos".** Editorial RA-MA. 2003
- **Metodologias Agiles dentro de la empresa. Ultima visita 19-09-2019** **https://www.iebschool.com/blog/que-son-metodologias-agiles-agile-scrum/**
- **Holzner, S.: "La biblia de JAVA 2".** Editorial Anaya Multimedia 2000.
- **Casado Iglesias, C.: "C.F.G.S Entornos de desarrollo"** Editorial RA-MA. 2012
- **Wikipedia, la enciclopedia libre. http://es.wikipedia.org/** Última visita: Octubre 2018.
- **López, J.C.: "Curso de JAVA http://www.cursodejava.com.mx** Última visita: Octubre 2018.
- **Documentación oficial Java JSE 8 http://docs.oracle.com/javase/8/** Última visita: Octubre 2015.
- **Programación en castellano: Java. http://www.programacion.net/java Última visita: Octubre 2015.**